

计算机体系结构

新型计算机研究所

陈 衡

西一楼 A 段415室

Email: hengchen@xjtu.edu.cn

QQ群号: 1031141115

第3章 流水线技术

- ◆ 3.0 先行控制技术
- ◆ 3.1 流水线的基本概念
- ◆ 3.2 流水线的性能指标
- ◆ 3.3 非线性流水线的调度
- ◆ 3.4 流水线的相关与冲突
- ◆ 3.5 流水线的实现

提高计算机性能的三种途径

◆ 1、提高处理机主频

- 50—60年代采用的主要技术，每3—4年提高一个数量级

◆ 2、采用更好的算法和设计更好的功能部件

- RISC，多位乘/除法，ROM乘法器等

◆ 3、采用指令级并行技术：三种

- **流水处理机**，超流水处理机（superpipelining）

- 指令执行部件采用流水线技术。
- 8段或8段以上流水线处理机称为超流水处理机。

- 超标量处理机（superscalar）技术

- 一个处理机中设置多个独立的功能部件

- 超长指令字（very long instruction word, VLIW）

- 在一条指令中，设置有多个独立的操作字段，每个字段可以独立控制各功能部件并行工作。

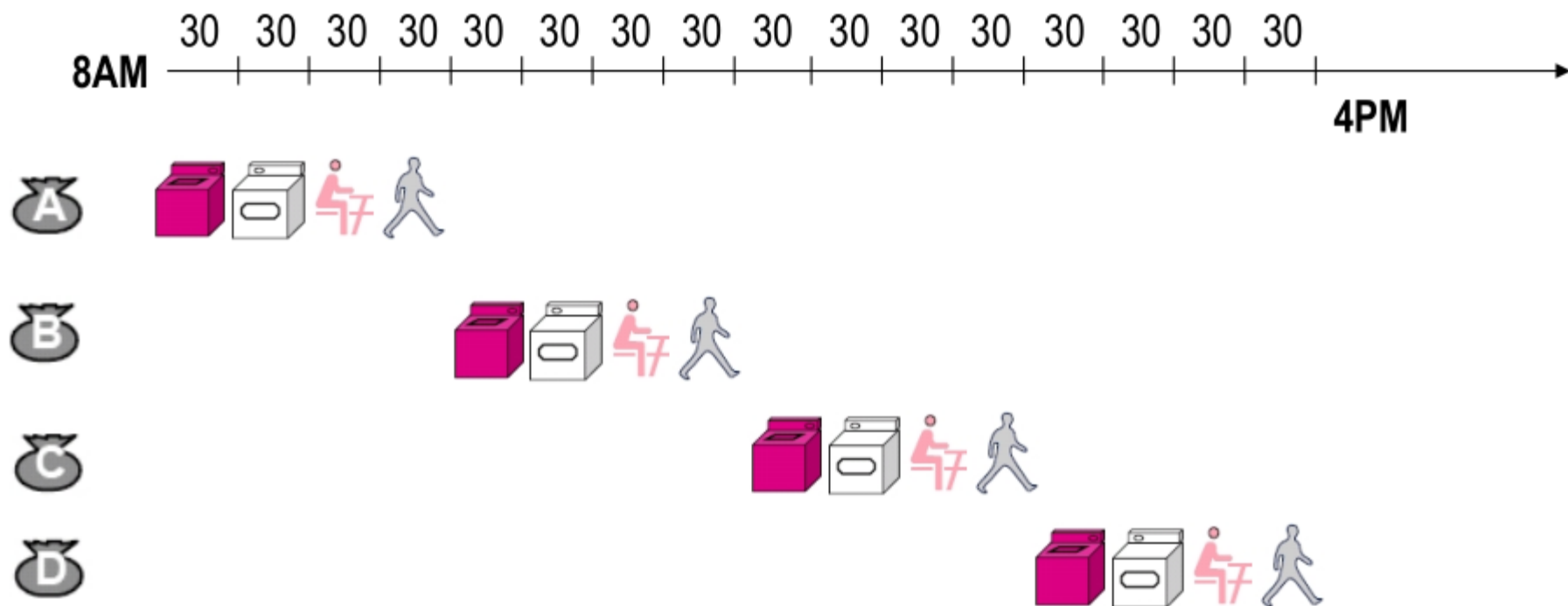
以洗衣店为例

◆ 工作需求和背景

- A、B、C、D每人有一包衣服
需要洗涤、烘干、熨整
- 洗衣机需要30分钟
- 烘干机需要30分钟
- 熨斗需要30分钟
- 洗衣工需要30分钟将衣物放到
抽屉里

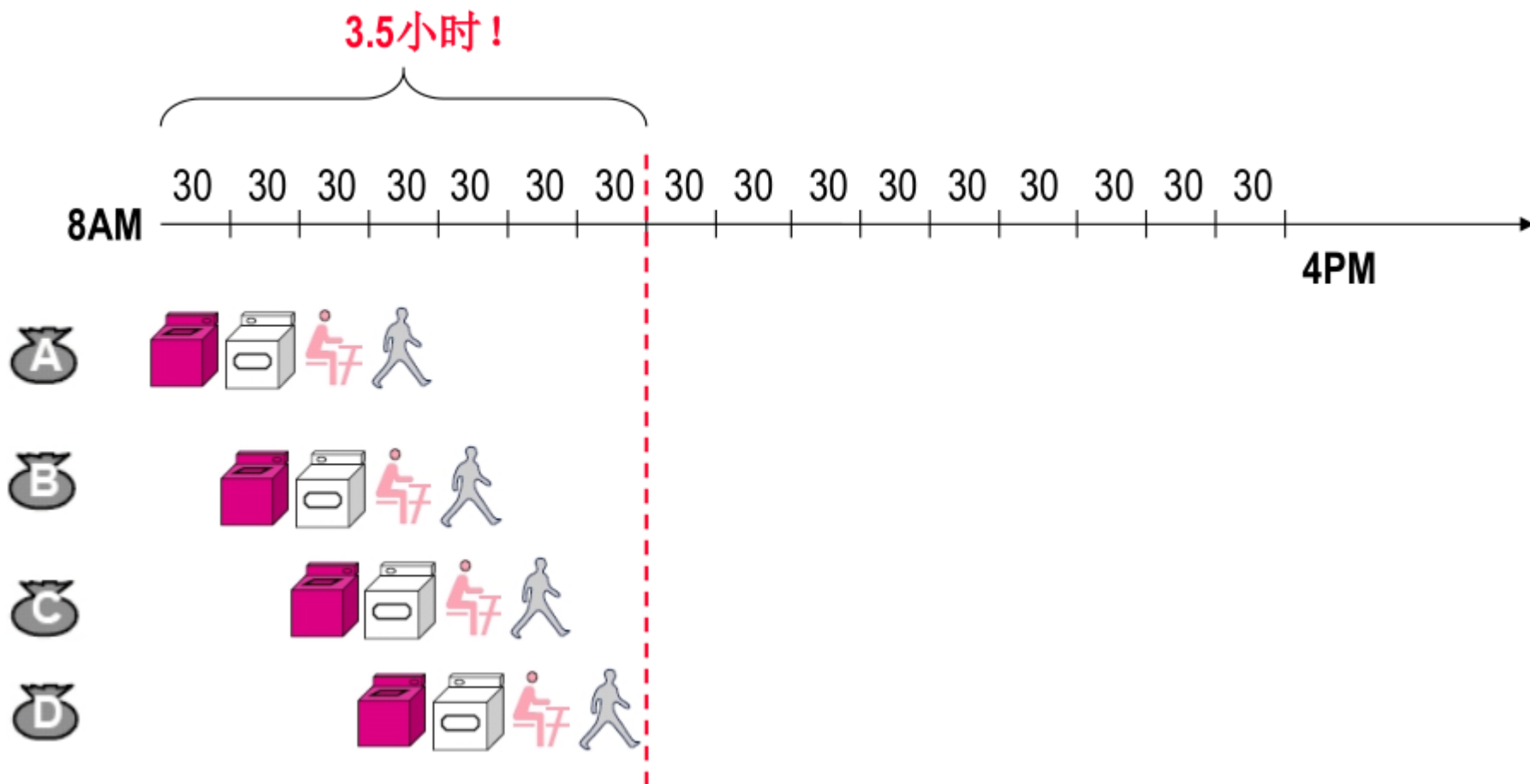


串行洗衣店



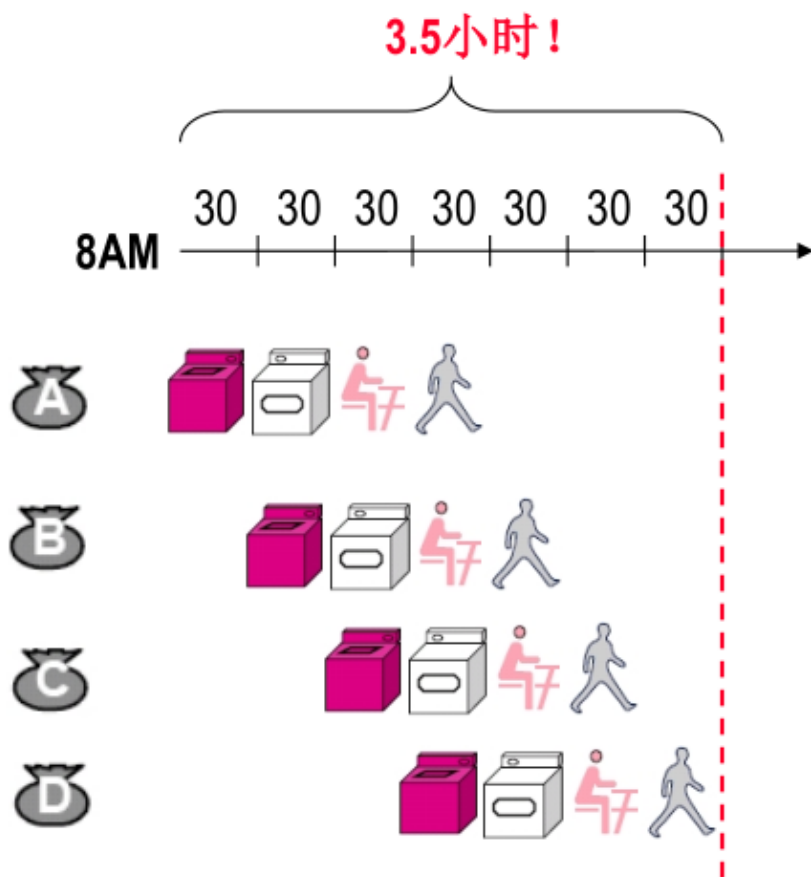
- 串行洗衣店需要8个小时完成4包衣服的工作量
- 如果采用流水线技术，需要多久？

流水化洗衣店



◆ 假设有4个工人

流水线的性质

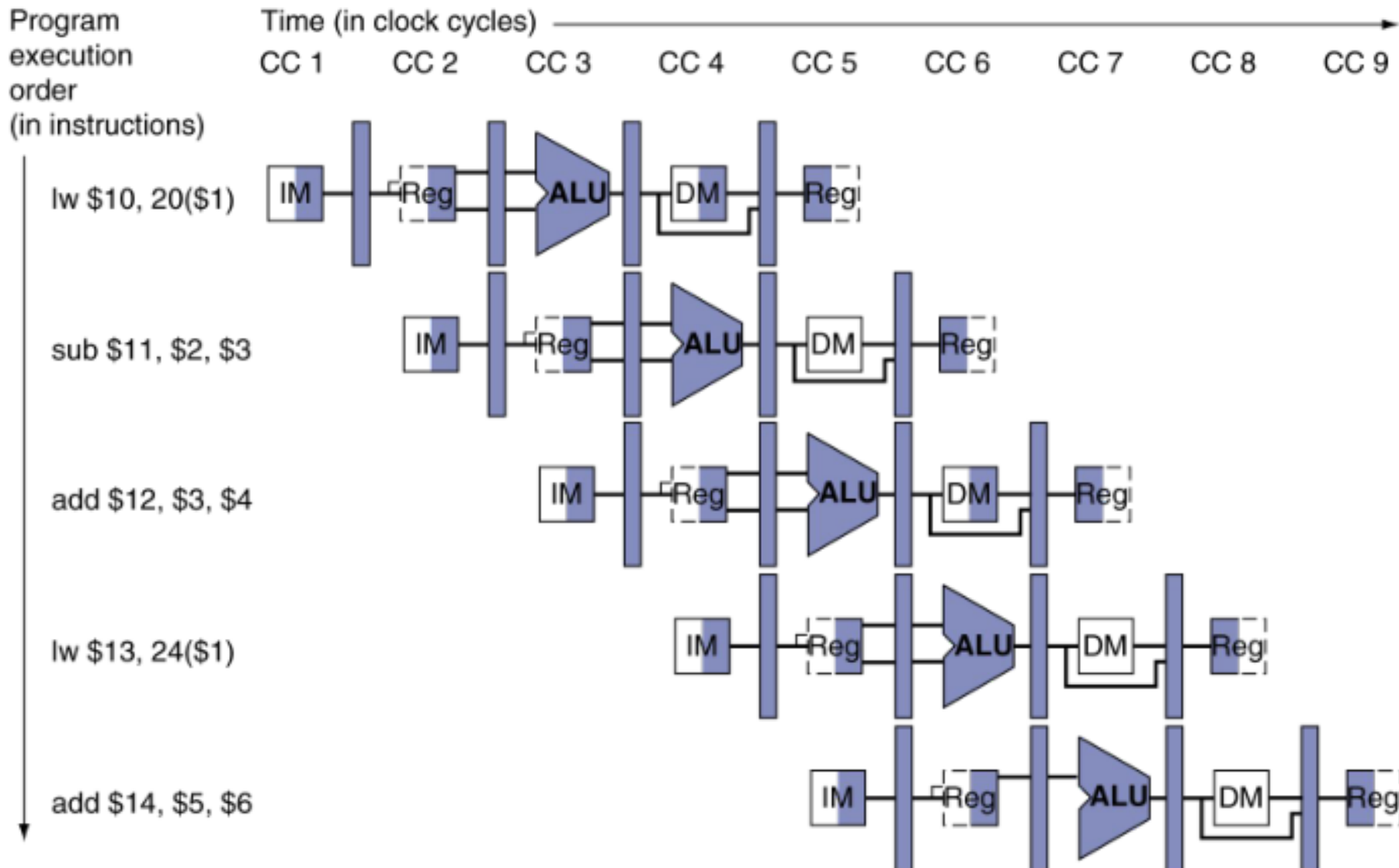


- 流水技术无助于减少单个任务的**处理延迟(latency)**，但有助于提高整体工作负载的**吞吐率(throughput)**
- **多个**不同任务同时操作, 使用**不同**资源
- 潜在加速比= **流水线级数**
- 流水线的速率受限于**最慢**的流水段
- 开始**填充**流水线的时间和最后**排放**流水线的时间降低加速比
- **相关**将导致流水线暂停

LOAD指令执行的步骤

- IF: 取指
 - 从指令存储器中取出指令
- ID: 取操作数和译码
 - 取寄存器和指令译码
- EX: 执行
 - 计算存储器的地址
- MEM: 存储器访问
 - 从数据存储器中读取数据
- WB: 写回
 - 将数据写回到寄存器堆

指令流水化执行



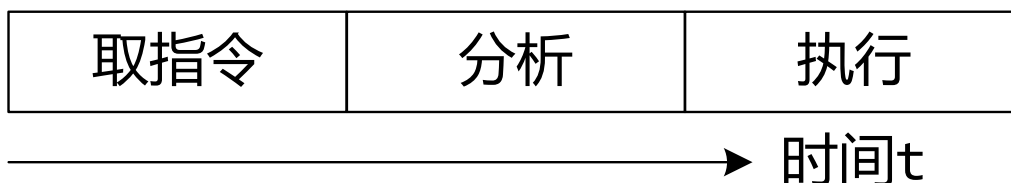
3.0 先行控制技术

- ◆ 先行控制（look-ahead）技术目的和作用：
 - 在实现指令的重叠执行时，各部件尽量连续、并行的工作的需要
 - 采用先行控制技术保证功能部件级流水线的饱满
- ◆ 先行控制（look-ahead）技术：
 - 通过对**指令流**和**数据流**的**预处理**和**缓冲**，使指令分析器和执行部件尽量连续并行工作。
- ◆ 先行控制技术性质：
 - 先行控制是流水线处理机和超标量处理机的基础。
 - **先行控制技术 = 缓冲技术 + 预处理技术。**
- ◆ 基本思路：
 - 通过指令的重叠执行，提高系统的性能。

3.0 先行控制技术

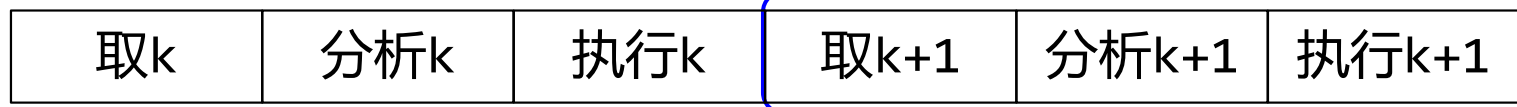
◆ 分析前提假设：

- 指令执行过程分为**取指令**，**指令分析**，**指令执行**



◆ 多条指令的执行方式：

◆ 1) 顺序执行方式



- n条指令时间：
$$T = \sum_{i=1}^n (t_{\text{取指令}i} + t_{\text{分析}i} + t_{\text{执行}i})$$

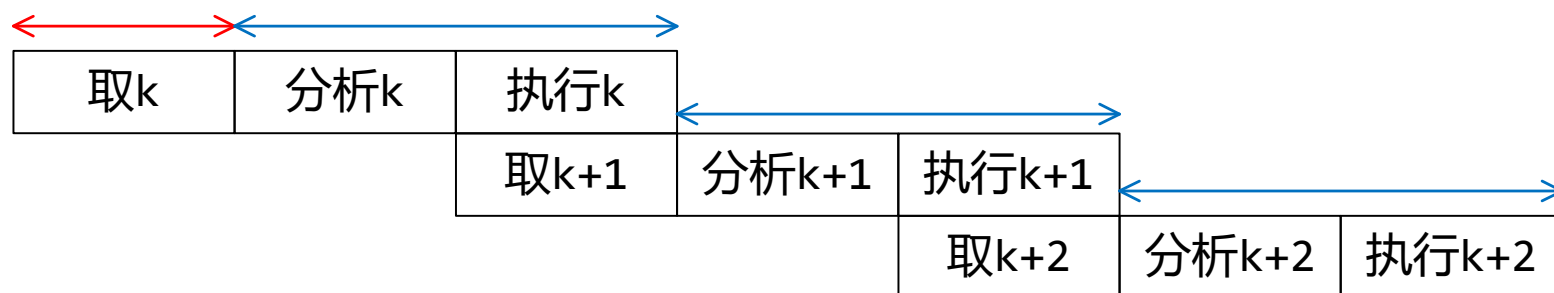
- 当 $t_{\text{取}} = t_{\text{分}} = t_{\text{执}} = t$ 时， $T = 3nt$

- 优点：控制简单
- 缺点：速度慢，功能部件利用率低

3.0 先行控制技术

◆ 2) 一次重叠 (Overlap) 执行方式

- 上一条指令的执行阶段与下一条指令的取指阶段完全重叠



- n 条指令时间:

$$T = (1 + 2n)t$$

- 优点:

- 执行时间缩短, 减少1/3
- 部件利用率提高

- 缺点: 需要增加硬件, 增加一个指令寄存器??

3.0 先行控制技术

◆ 3) 二次重叠执行方式

- 将相邻两条指令的重叠时间再往前提前一个阶段

取k	分析k	执行k		
	取k+1	分析k+1	执行k+1	
		取k+2	分析k+2	执行k+2

- n条指令时间：

$$T = (2 + n)t$$

- 优点：

- 执行时间缩短，减少2/3

- 部件利用率最高

- 缺点：需要增加更多控制硬件

先行控制方式的原理和结构

- ◆ 由于**各功能部件的延时**不同，为了实现各功能段平滑连续工作，提高功能部件利用率，实现系统的高吞吐率，就要保证各功能部件并行、连续的平滑工作，必须**采用先行控制方式**，是实现流水线处理机的基础。
- ◆ 重叠与流水的关系
 - 最理想的深度的重叠就是流水工作方式
- ◆ 实现指令重叠执行必须解决的问题：
 - 如何才能使取指、分析和执行同时进行？平滑连续不断地工作？
 - **同时进行**：独立控制、并可以同时访问RAM
 - **平滑连续不断**：预取、预处理、缓冲、消除相关

先行控制方式的原理和结构

- ◆ 处理器中需要**独立的取指、分析和执行部件**
 - 需要把顺序执行方式中的一个集中的指令控制器，分解成三个相对独立的控制器，即：**存控、指控、运控**。
- ◆ 解决访问RAM的冲突问题
 - 实现指控、运控在存控的管理下可同时并行进行取指、读数、写数
 - 指令和操作数独立编址
 - 采用**低位交叉存取**
 - 采用先行控制技术（缓冲技术+预处理技术），**普遍采用**
 - ① **缓冲技术**：在工作速度不固定的两个功能部件之间设置缓冲栈，以平滑他们间的速度差，处理机中有四个缓冲栈。
 - ② **预处理技术**：将指令转成R-R型，在进入ALU之前准备好所需要的全部数据。

先行控制方式中的重叠执行

- ◆ 将指令分析器需要的指令事先取到指令缓冲栈中，能够实现取指令，分析指令，执行指令重叠。
- ◆ **每次取指令在先行缓冲栈中**得到，时间很短，因此可把取指令、分析指令合并到一起，一次重叠执行为：

分析k	执行k		
	分析k+1	执行k+1	
		分析k+2	执行k+2

- ◆ 理论上 n 条指令时间：

$$T = (1 + n)t$$

指令重叠执行需解决的问题

◆ 结构相关：

- 指令的复杂性，导致“分析”、“执行”指令所需要的时间往往相差很大，从而导致相互等待

◆ 数据相关：

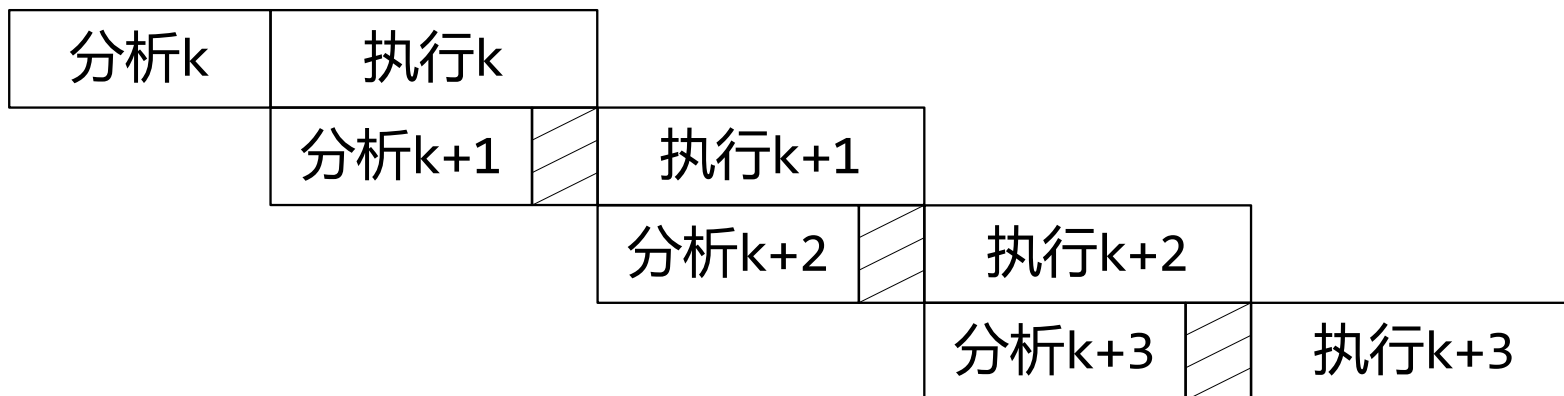
- 存在主存操作数相关、通用寄存器相关、指令相关、变址相关的问题，从而导致指令分析、执行无法完全重叠执行

◆ 控制相关：

- 存在转移、子程序调用等情况，从而导致指令缓冲器中预取的指令和已分析完的指令作废

指令执行时序分析

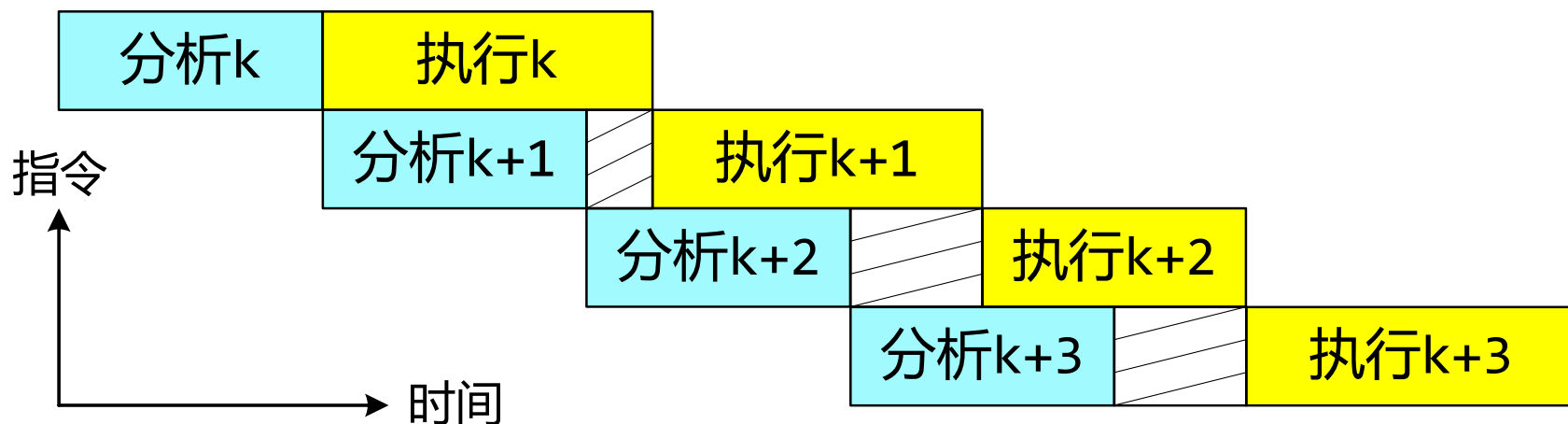
- ◆ 非先行控制方式中，分析指令和执行指令时间不相等时的一次重叠执行方式



- ◆ 由于时间不相等各功能部件不能一直处于忙碌状态，**分析器要等待**，执行部件可能会有空闲，即结构相关问题

指令执行时序分析

- ◆ 如何解决各功能部件不能一直处于忙碌状态问题？
 - 结构相关解决方法：**功能部件之间设置缓冲栈**，指令分析器和指令执行部件就能完全独立、平滑连续的工作



采用先行缓冲栈的指令执行过程

指令执行时序分析



采用先行缓冲栈时指令执行过程的一种表示方法

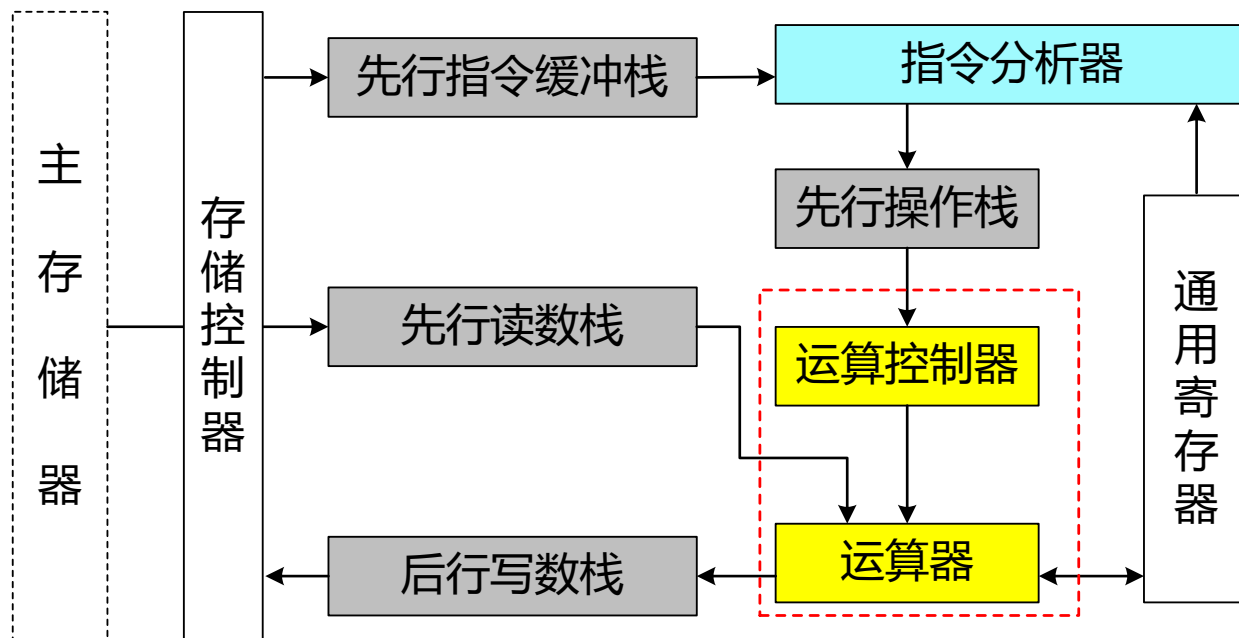
$$T_{\text{先行}} = t_{\text{分析}_1} + \sum_{i=1}^n t_{\text{执行}_i} \approx \sum_{i=1}^n t_{\text{执行}_i}$$

- ◆ 程序中，由于分析阶段有些指令就已被执行，不进入执行阶段，因此，分析指令的条数 > 执行指令的条数，可以提高整体处理速度

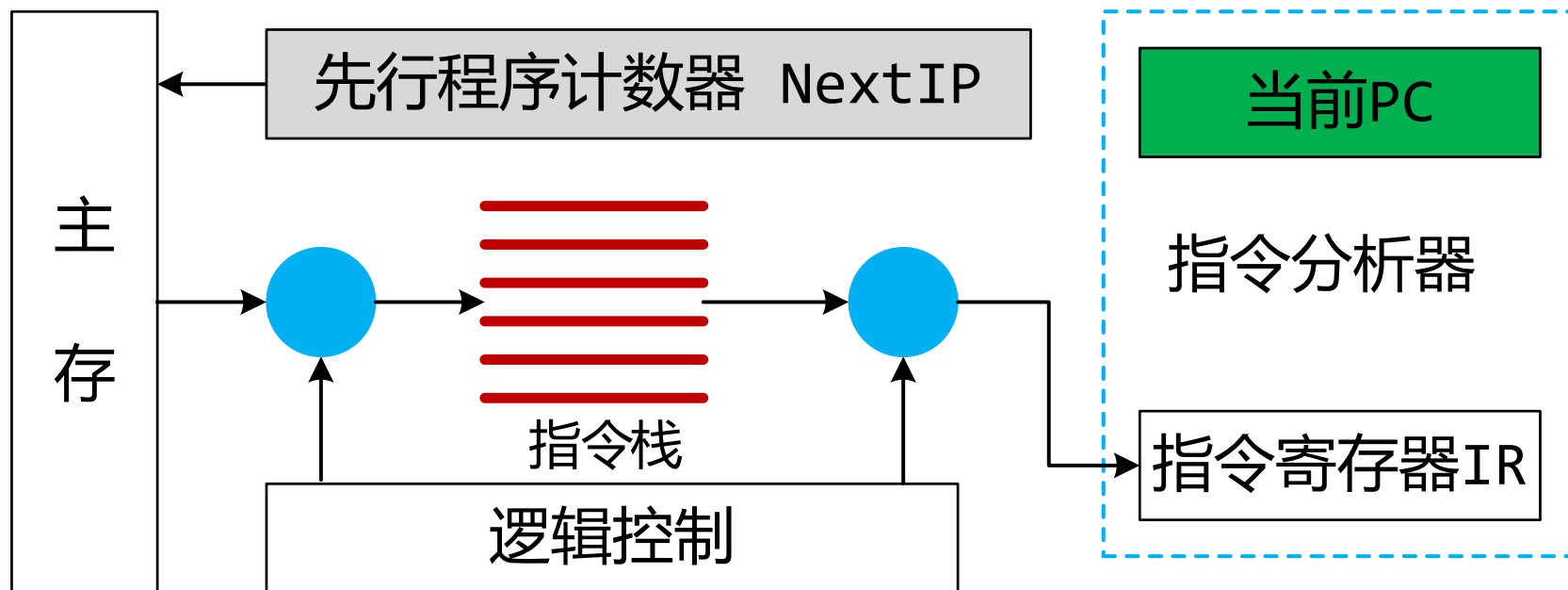
先行缓冲栈

◆ 先行控制中的结构相关问题解决策略

- 先行指令缓冲栈：主存和指令分析器之间
- 先行操作栈：指令分析器和控制器之间
- 先行读数栈：主存和运算器之间
- 后行写数栈：主存和运算器之间



先行控制方式的处理器结构



◆ 缓冲站1：先行指令缓冲栈的结构

- ① 存储控制器将取指令安排在最低优先级，其余依次为读操作数和写结果
- ② 先行指令缓冲栈是利用RAM的空闲时间进行取指操作。

指令分析器

- ◆ 对已经存放在先行指令缓冲栈里的指令进行预处理，再送入**先行操作栈**。具体预处理为：
 - 跳转及条件转移指令：在指令分析器中直接执行
 - 运算型指令、移位指令、数据传输指令等，先转成R-R型指令，然后再送入先行操作栈，称为RR*指令
 - 变址型（RX）指令和存储器指令（RS）指令，将EA指令送入先行读数站，再生成RR*指令。

指令分析器-举例

◆ 例如: Add AX, [200H+BX]

- Step 1: 分析器计算 $EA = BX + 200H$
- Step 2: 分析器将 $EA \rightarrow$ 先行读数栈
- Step 3: 先行读数栈通过存控从RAM中取数, 并放在先行读数栈寄存器 R^*
- Step 4: 分析器将用先行读数栈寄存器 R^* 编码替代 $[200H+BX]$
- Step 5: 形成 RR^* 指令: Add AX, R^*
- Step 6: 将形成的 RR^* 指令送入先行操作栈

先行控制方式的处理机结构

◆ 缓冲站2：先行操作栈

- 是指令分析器和运算控制器之间的缓冲，从而使得两者可以并行独立运行
- 采用FIFO的方法
- 为什么叫先行操作栈？相对于正在执行的指令而言，存放在其内的RR*型指令是后续指令“先行”预处理的结果。

◆ 缓冲站3：先行读数栈

- 组成：由一组缓冲Registers和相关控制逻辑组成，每个Register由地址、操作段和标志字段三部分组成。
- 作用：主存和ALU之间的缓冲存储器，用于平滑两者之间的速度差。
- 为什么叫先行读数栈？相对于正在执行的指令而言，是把后续指令的操作数提前读取到其中。

先行控制方式的处理器结构

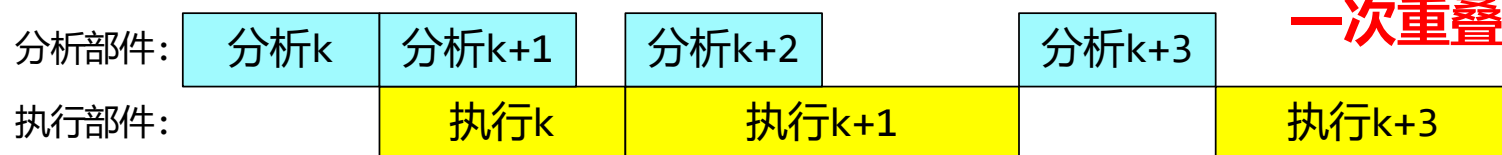
◆ 缓冲站4：后行缓冲写数栈

- 背景：当指令中出现“写”MM指令时
- 作用：平滑MM和ALU之间的速度差
- 组成：和先行读数栈类同
- 为什么叫后行写数栈？
 - 相对于正在执行的指令而言，是把先前执行指令的结果写回到MM。

先行控制技术总结

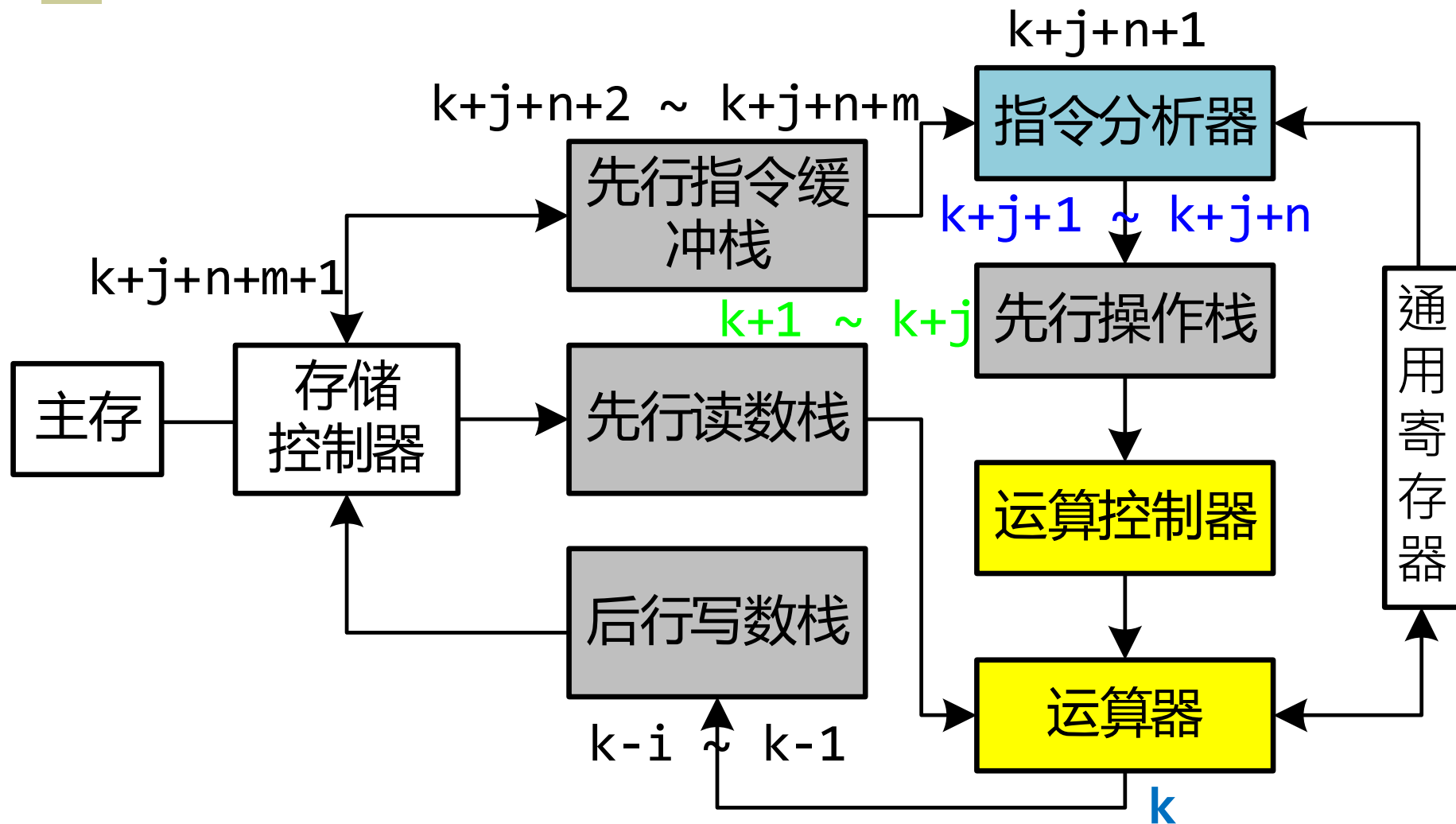
- ◆ 先行控制技术 = 预处理技术 + 缓冲技术
- ◆ 方法：通过对指令流和数据流的预处理和缓冲，使得指令分析和指令执行部件能够完全独立、平滑连续地工作。
- ◆ 先行控制和一次重叠不同在于：先行控制不要求分析和执行的两条指令相邻，而一次重叠则必须相邻。**先行控制并行性更高。**

假设k+2
是跳转指令



- ◆ 在系统结构中，通常把先行指令缓冲栈、先行读数栈、先行操作栈和后行写数栈称为**先行控制器**。
- ◆ 先行控制器和指令分析器一起构成指令控制部件。

指令执行情况分析-举例



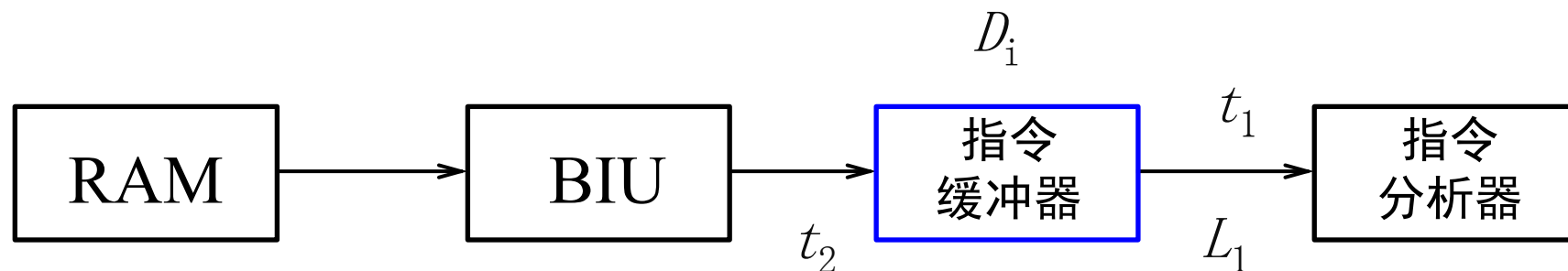
◆ 假设ALU正在执行第 k 条指令

先行控制技术下程序执行情况

- ① 设第 $k-i-1$ 条指令已经执行完毕;
- ② 第 $k-i \sim k-1$ 共 i 条指令, 在后行写数据栈中等待把结果写回主存;
- ③ 第 k 条指令正在ALU中执行;
- ④ 第 $k+1 \sim k+j$ 共 j 条指令, 已经由指令分析器预处理完成, 以RR*指令存放在先行操作栈之中, 而且操作数也已经在“先行读数栈”中。
- ⑤ 第 $k+j+1 \sim k+j+n$ 共 n 条指令, 已经生成RR*指令在先行操作栈之中, 而操作数尚未取到“先行读数栈”之中。
- ⑥ 第 $k+j+n+1$ 指令, 正在“指令分析器”之中进行预处理。
- ⑦ 第 $k+j+n+2 \sim k+j+m$ 共 $m-1$ 条指令, 已经从主存预取到“先行指令缓冲栈”之中。
- ⑧ 下一条第 $k+j+n+m+1$ 指令, 准备从主存读取到CPU之中的指令。

缓冲深度

- ◆ 核心问题：各缓冲栈的register个数设置多少合适？即所谓“缓冲深度”是多少？
 - 缓冲深度= register个数
 - 少了，起不到平滑作用
 - 多了，浪费，控制复杂，实现成本高
- ◆ 缓冲深度计算和分析方法
 - 静态分析法：进行极端情况分析



指令先行缓冲栈深度设计

- ◆ 静态分析：分析两种极端情况来计算指令先行缓冲栈深度
- ◆ 1、缓冲栈满→缓冲栈空：分析器这时**分析**的某类指令**速度很快**，从主存**预取指令很慢**。

- 假设：缓冲深度 D_i ，指令分析时间 t_1 ，从主存取指令时间 t_2 ($t_2 > t_1$)，分析的指令条数 L_1 。那么从缓冲栈满到取空满足：

$$L_1 t_1 = (L_1 - D_i) t_2$$

$$\text{即 } D_i = \left\lceil \frac{L_1 \cdot (t_2 - t_1)}{t_2} \right\rceil$$

- 当这种指令流的连续长度大于 L_1 条时，指令先行缓冲栈将被取空，指令分析器将处于等待。

指令先行缓冲栈深度设计

- ◆ 2、缓冲栈空→缓冲栈满：分析器这时**分析**的某类指令**速度很慢**，从主存**取指令相对很快**。

- 假设：缓冲深度 D_i ，指令分析时间 t_1' ，从主存取指令时间 t_2' ，从主存取到的指令条数 L_2 ，分析的指令条数 $L_2 - D_i$ 。那么从缓冲栈空到取满满足：

$$(L_2 - D_i)t_1' = L_2 t_2'$$

$$\text{即 } D_i = \left\lceil \frac{L_2 \cdot (t_1' - t_2')}{t_1'} \right\rceil$$

- 当这种指令流的连续长度大于 L_2 条时，指令先行缓冲栈将被充满，先行缓冲栈的作用减弱。

指令先行缓冲栈深度-举例

◆ 一般用**第一种情况**计算先行缓冲栈深度

- 假设这种指令流的平均长度=9，即10条连续指令中，90%的指令是执行时间短的指令；
- 指令分析时间 $t_1=1$ 个机器周期；
- 从主存取指令时间 $t_2=4$ 个机器周期
- 那么缓冲深度 D_i 应为：

$$D_i = \left\lceil \frac{L_1 \cdot (t_2 - t_1)}{t_2} \right\rceil = \left\lceil \frac{9 \cdot (4 - 1)}{4} \right\rceil = 7$$

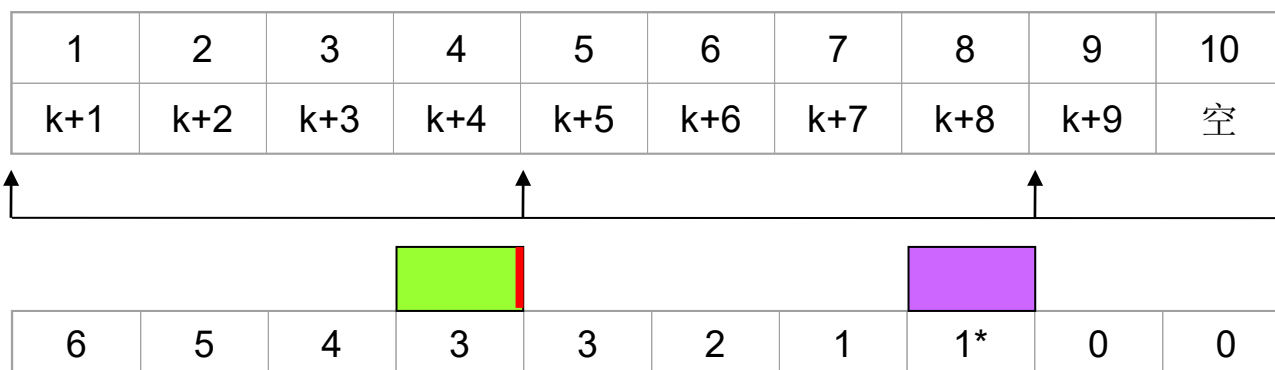
工作时间关系：

工作周期

指令分析器的指令序列
向主存请求取指令

指令取到先行缓冲中

缓冲栈中剩余指令条数



指令先行缓冲栈深度-举例

- ◆ 增加存储读写字宽，Cache容量和级数（缩短了数据访问周期），有利于减少缓冲深度。
- ◆ 同一台处理机中缓冲栈的深度关系：
 - $D_i \geq D_c \geq D_r \geq D_w$
 - IBM 370/165机, $D_i = 4$, $D_c = 3$, $D_r = 2$, $D_w = 1$

3.1 流水线的基本概念

◆ 什么是流水线技术？

- 把一个重复的过程**分解**为若干个子过程，每个子过程由专门的功能部件来实现。
- 把**多个处理过程在时间上错开**，依次通过各功能段，这样，每个子过程就可以与其他的子过程并行进行。

◆ 流水线的级或段

- 流水线中的每个子过程及其功能部件称为流水线的**级或段**，**段与段相互连接形成流水线**。

◆ 流水线的段数称为**流水线的深度**。

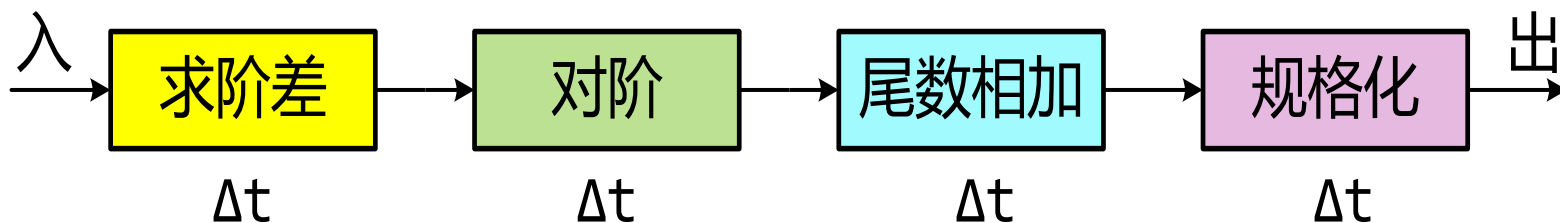
◆ 指令流水线

- 把指令的解释过程分解为 n 个子过程，并让这 n 个子过程分别用独立的分析部件和执行部件来实现。

指令流水线

◆ 浮点加法流水线

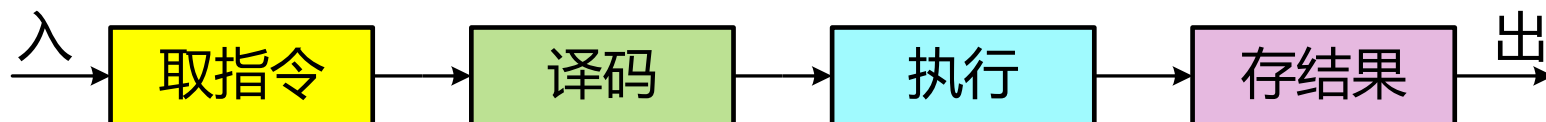
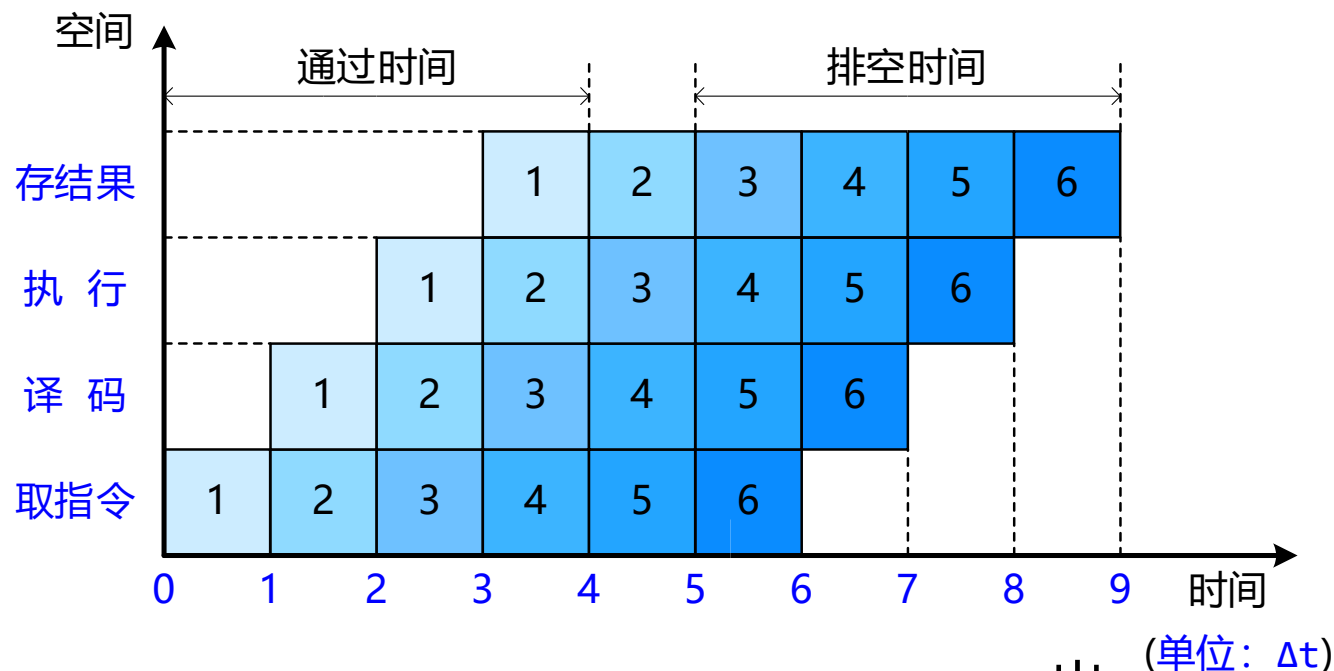
- 把流水线技术应用于运算的执行过程，就形成了运算操作流水线，也称为部件级流水线。
- 把浮点加法的全过程分解为**求阶差**、**对阶**、**尾数相加**、**规格化**4个子过程。



时空图

- 时空图从**时间**和**空间**两个方面描述了流水线的工作过程。时空图中，横坐标代表时间，纵坐标代表流水线的各个段。

- 4段指令流水线的时空图

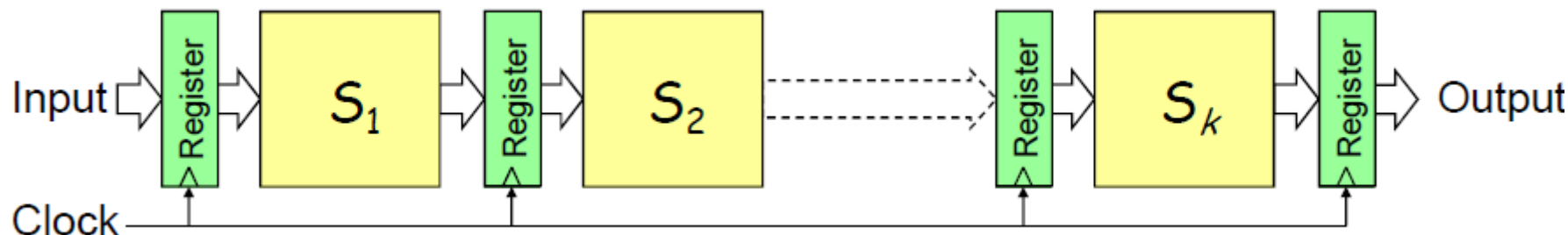


流水线的技术特点

- ◆ 流水线把一个处理过程分解为若干个子过程（段），每个子过程由一个专门的功能部件来实现。
- ◆ 流水线中各段的时间应尽可能相等，否则将引起**流水线堵塞、断流**。
 - 时间长的段将成为流水线的**瓶颈**。
- ◆ 流水线每一个功能部件的后面都要有一个缓冲寄存器（锁存器），称为**流水寄存器**。
 - 作用：在相邻的两段之间传送数据，以保证提供后面要用到的数据，并把各段的处理工作相互隔离。
- ◆ 流水技术适合于大量重复的时序过程，只有在输入端不断地提供任务，才能充分发挥流水线的效率。
 - 流水线需要有**通过时间**和**排空时间**

同步流水线

- ◆ 流水段之间采用时钟控制的寄存器文件 (clocked registers)
- ◆ 时钟上升沿到达时，所有寄存器**同时**保存前一流水段的结果
- ◆ 流水段是组合逻辑电路
- ◆ 流水线设计中希望各段**相对平衡**，即所有段的延迟时间大致相等
- ◆ **时钟周期取决于延迟最长的流水段**



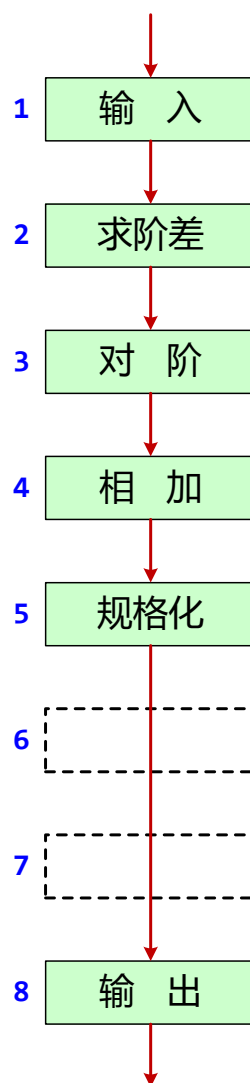
3.1.2 流水线的分类

- ◆ 从不同的角度和观点，流水线可分为不同的种类。
- ◆ 按功能分
 - **单功能流水线**：只能完成一种固定功能的流水线
 - Pentium中有一条5段的整数运算流水线和8段的浮点运算流水线，我国YH-1有18条单功能流水线。
 - **多功能流水线**：流水线的各段可以进行不同的连接，以实现不同的功能。
 - Texas公司的ASC(Advanced Scientific Computer)中采用的4条8段流水线。

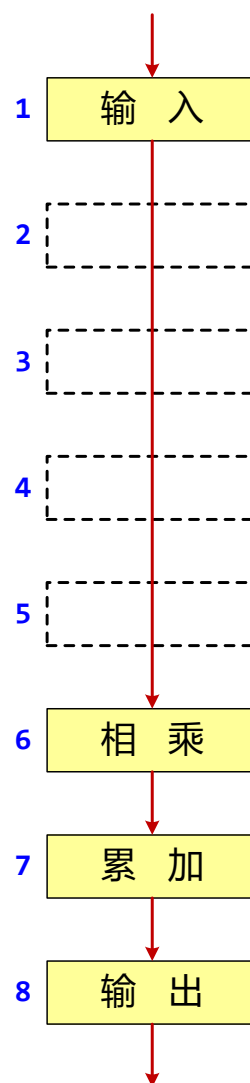
流水线的分类



(a) 分段



(b) 浮点连接



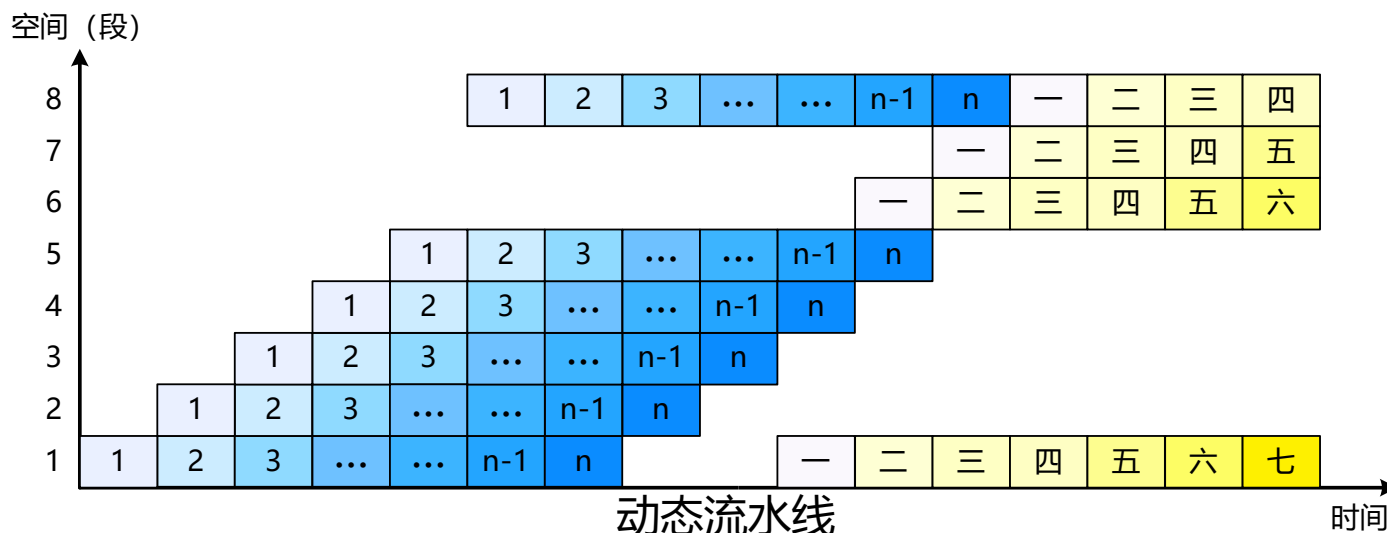
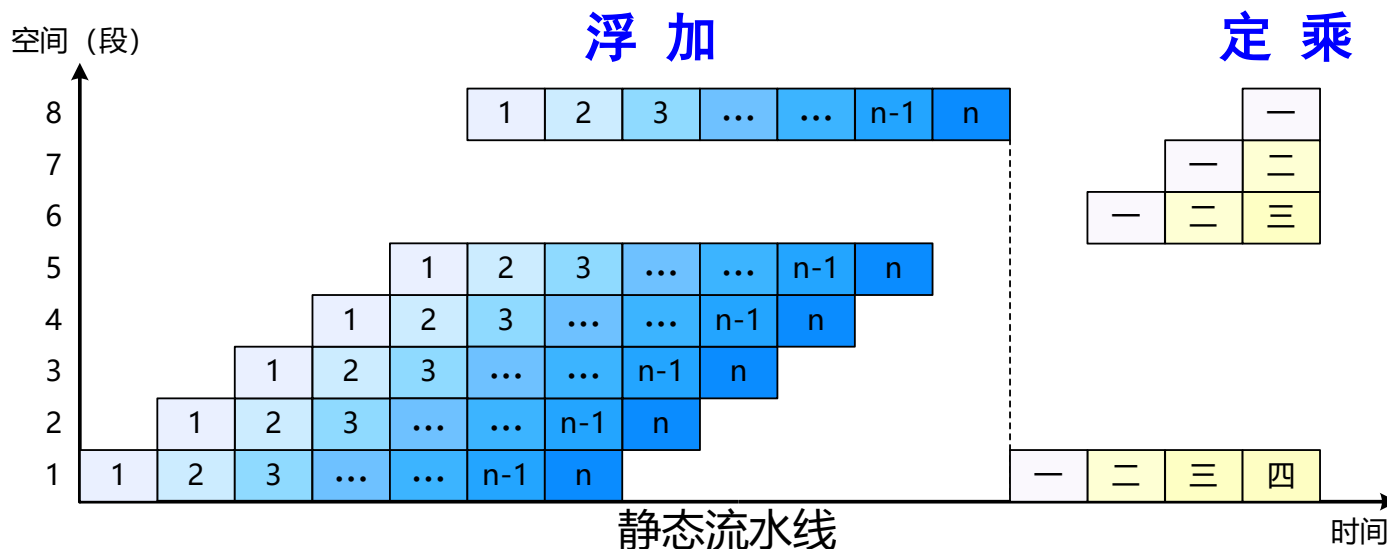
(c) 定乘连接

3.1.2 流水线的分类

◆ 按连接方式分

- 按照同一时间内各段之间的连接方式对多功能流水线做进一步的分类
- **静态流水线**：在同一时间内，多功能流水线中的各段只能按同一种功能的连接方式工作。
 - 对于静态流水线来说，只有当输入的是一串相同的运算任务时，流水的效率才能得到充分的发挥
 - 例如：ASC的8段流水线
- **动态流水线**：在同一时间内，多功能流水线中的各段可以按照不同的方式连接，同时执行多种功能。
 - 优点：灵活，能够提高流水线各段的使用率，从而提高处理速度。
 - 缺点：控制复杂。

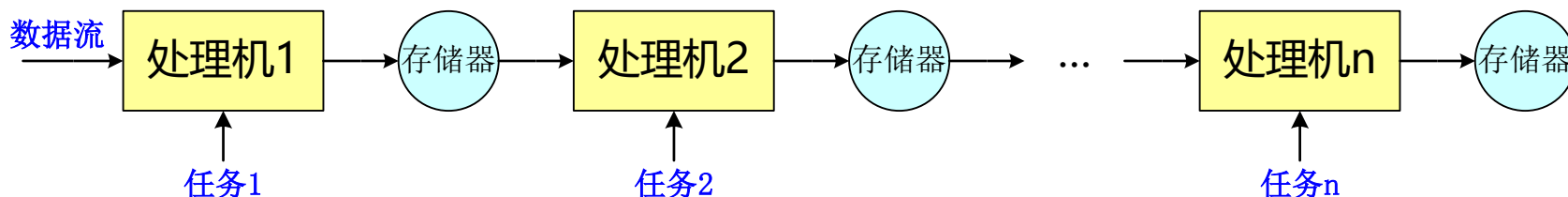
静、动态流水线时空图的对比



3.1.2 流水线的分类

◆ 按流水的级别分

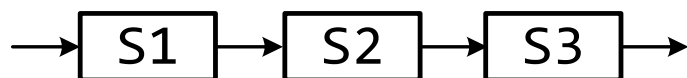
- **部件级流水线**（运算操作流水线）：把处理机的算术逻辑运算部件分段，使得各种类型的运算操作能够按流水方式进行
 - 例如：前面的浮点加法流水线
- **处理机级流水线**（指令流水线）：把指令的解释执行过程按照流水方式处理。把一条指令的执行过程分解为若干个子过程，每个子过程在独立的功能部件中执行。
 - 例如：前面的4段指令流水线
- **处理机间流水线**（宏流水线）：它是由两个或者两个以上的处理机串行连接起来，对同一数据流进行处理，每个处理机完成整个任务中的一部分。



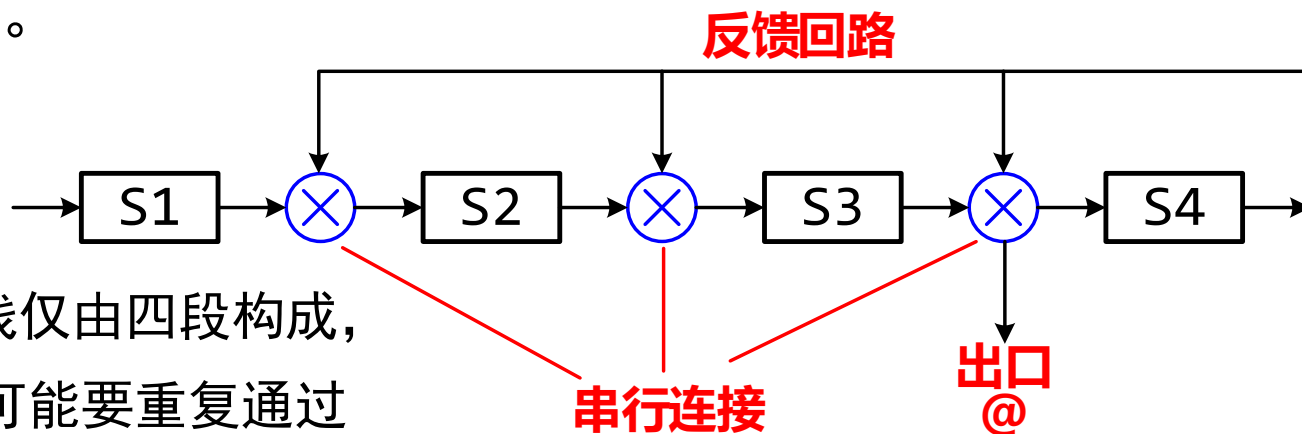
3.1.2 流水线的分类

◆ 按是否有反馈回路分

- **线性流水线**：流水线的各段串行连接，没有反馈回路。数据通过流水线中的各段时，每一个段最多只流过一次。



- **非线性流水线**：流水线中除了有串行的连接外，还有反馈回路。即存在个段多次执行的情况。适合于递归调用或多功能流水线。



- 虽然流水线仅由四段构成，但有些段可能要重复通过
- 例如任务@：

→ S1 → S2 → S3 → S4 → S2 → S3 → S4 → S3 →

3.1.2 流水线的分类

◆ 按输入输出任务次序分

- 根据任务流入和流出的顺序是否相同来进行分类
- **顺序流水线**：流水线输出端任务流出的顺序与输入端任务流入的顺序完全相同。每一个任务在流水线的各段中是一个跟着一个顺序流动的。
- **乱序流水线**：流水线输出端任务流出的顺序与输入端任务流入的顺序可以不同，允许后进入流水线的任务先完成（从输出端流出）。
 - 也称为无序流水线、错序流水线、异步流水线
 - 例如，在指令流水线中，有的指令要到主存储器中读操作数，也有的指令不需要到主存储器中读操作数。因此，在不发生数据相关的情况下，只要指令执行部件有空闲，不需要到主存储器中读操作数的指令可以超越需要等待从主存储器中读出操作数的指令先在指令执行部件中执行。

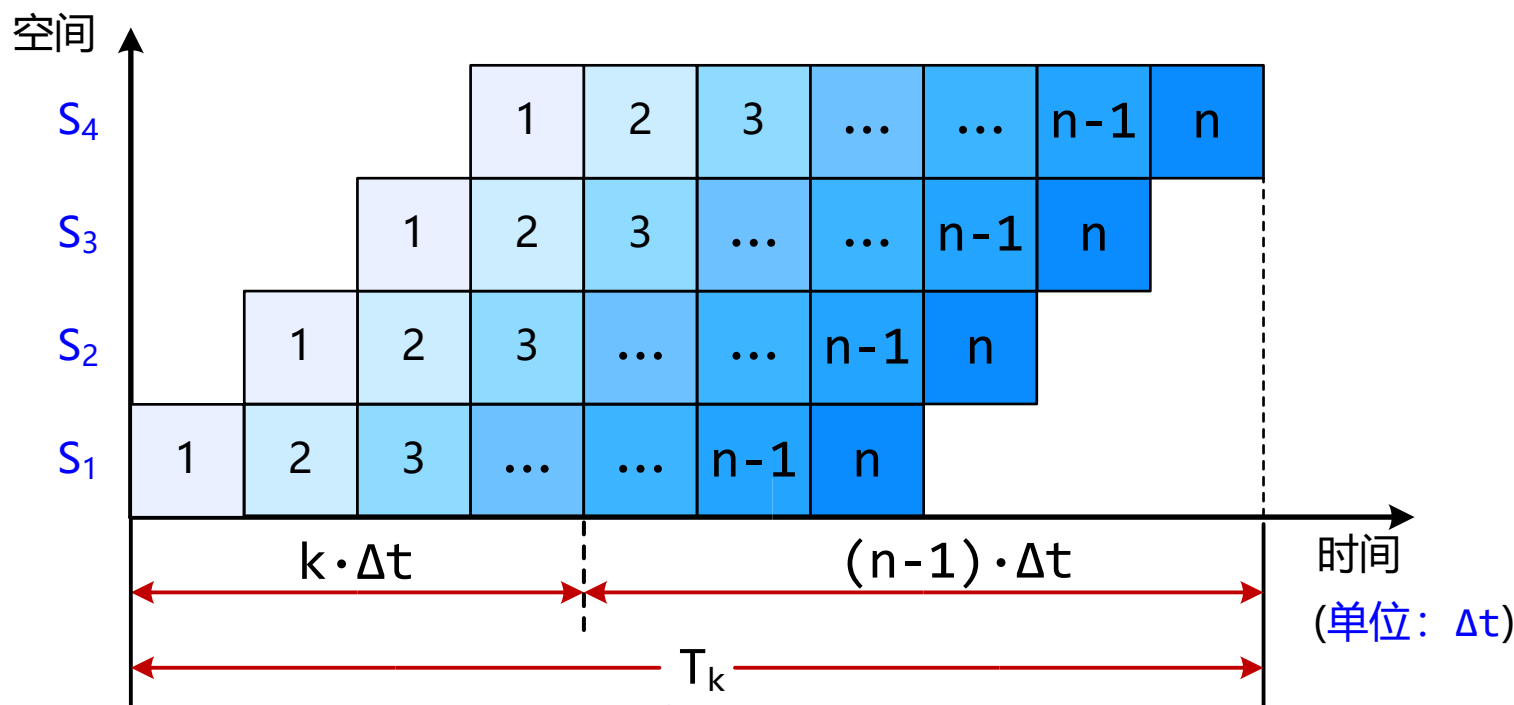
3.2 流水线的性能指标

◆ **吞吐率**：在单位时间内流水线所完成的任务数量或输出结果的数量。

$$TP = \frac{n}{T_K}$$

• n ：任务数

• T_k ：处理完成 n 个任务所用的时间



一条 k 段线性流水线时空图

吞吐率

- ◆ 流水线完成 n 个连续任务所需要的总时间为
 - 设流水线分 k 段，连续输入 n 个任务， Δt 为时钟周期，有

$$T_k = k\Delta t + (n - 1)\Delta t = (k + n - 1)\Delta t$$

- 可得到流水线的实际吞吐率 TP

$$TP = \frac{n}{(k + n - 1)\Delta t}$$

- 进一步可得该流水线的最大吞吐率

$$TP_{\max} = \lim_{n \rightarrow \infty} \frac{n}{(k + n - 1)\Delta t} = \frac{1}{\Delta t}$$

- 理解： $k \cdot \Delta t$ 为第一条指令的执行时间， $(n-1) \cdot \Delta t$ 为另外 $n-1$ 条指令执行所需要的时间。

吞吐率

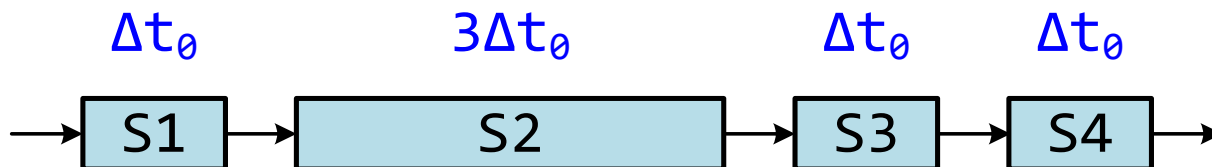
◆ 最大吞吐率与实际吞吐率的关系

$$TP = \frac{n}{k + n - 1} TP_{\max}$$

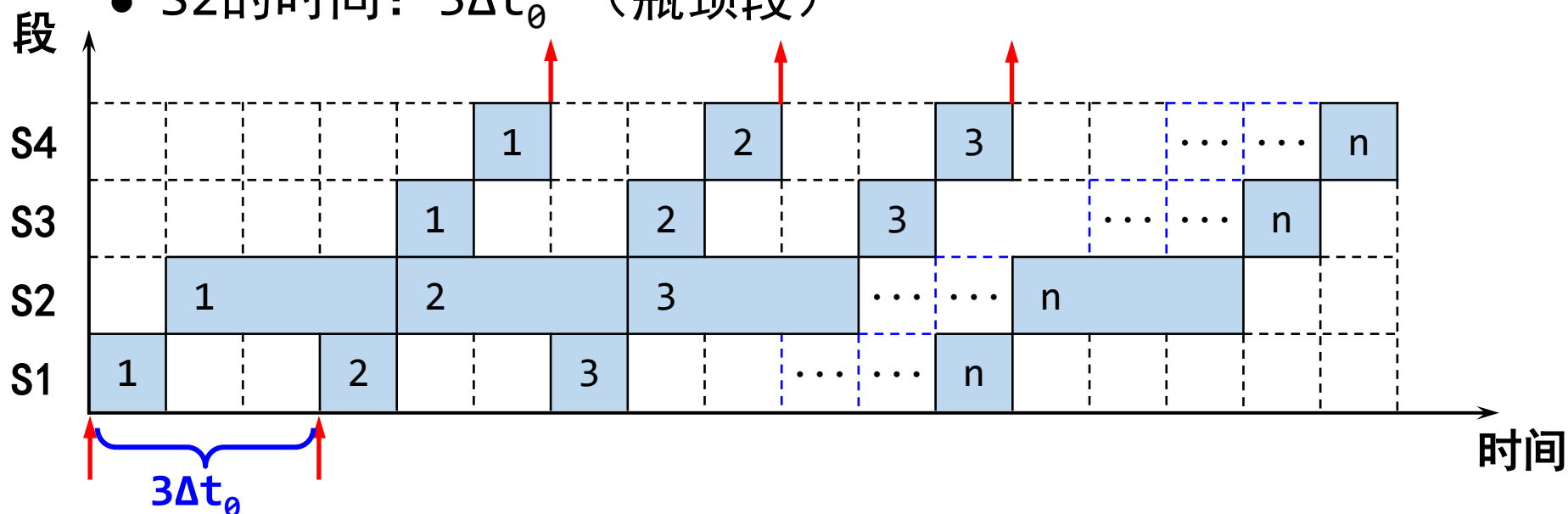
- 由于 $k > 1$ ，流水线的实际吞吐率 **小于** 最大吞吐率
- 它除了与每个段的时间有关外，还与流水线的段数 k 以及输入到流水线中的任务数 n 等有关。
- 只有当 $n \gg k$ 时，才有 $TP \approx TP_{\max}$ 。
- **说明：**
 - 适用于各段时间 Δt 均相等的线性流水线

各段时间不完全相等的流水线

◆ 各段时间不等的流水线及其时空图



- 一条4段的流水线
- S1, S3, S4各段的时间: Δt_0
- S2的时间: $3\Delta t_0$ (瓶颈段)



各段时间不完全相等的流水线

◆ 各段时间不等

的流水线的实际吞吐率：

- Δt_i 为第 i 段的时间，共有 k 个段

$$TP = \frac{n}{\sum_{i=1}^k \Delta t_i + (n-1) \max(\Delta t_1, \Delta t_2, \dots, \Delta t_k)}$$

◆ 流水线的最大吞吐率为

$$TP_{\max} = \frac{1}{\max(\Delta t_1, \Delta t_2, \dots, \Delta t_k)}$$

- 流水线的最大吞吐率与实际吞吐率主要是由**最长功能段时间**决定的，这个功能段就成了整个流水线的“**瓶颈**”。
- “瓶颈”造成的问题：资源浪费，除了流水线中的瓶颈段一直忙碌之外其余各段有许多是空闲的。

举例

- ◆ 例如：一条4段的流水线中，S1，S2，S4各段的时间都是 Δt ，唯有S3的时间是 $3\Delta t$ 。求其最大吞吐率

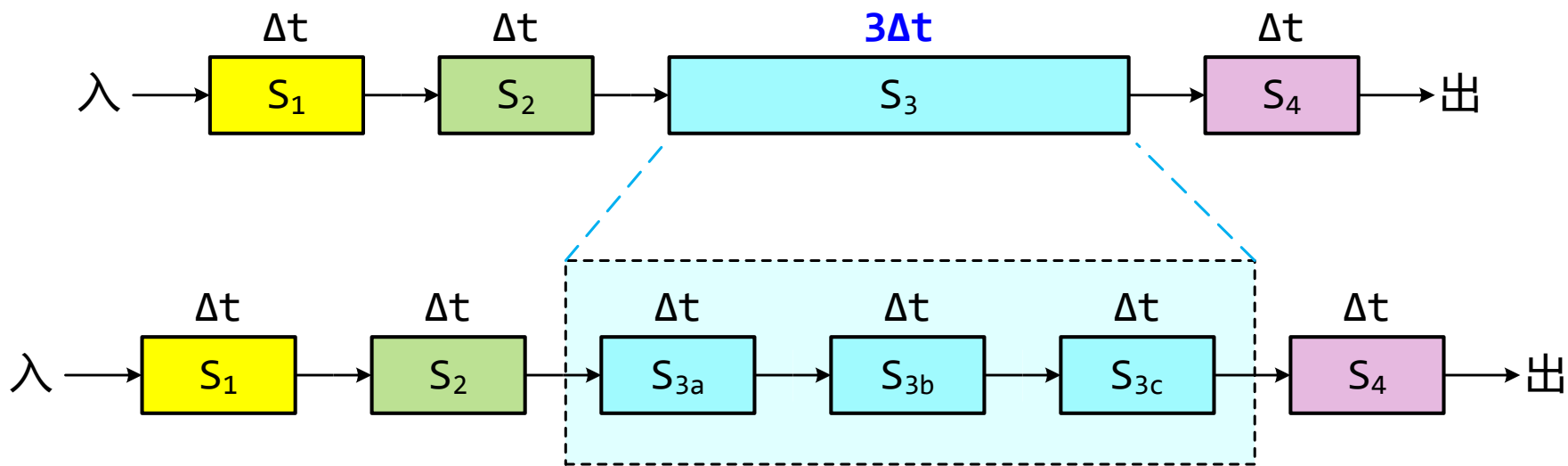


$$TP_{\max} = \frac{1}{\max(\Delta t, \Delta t, 3\Delta t, \Delta t)} = \frac{1}{3\Delta t}$$

流水线“瓶颈”问题的解决方法

- ◆ 方法1: 细分流水线的“瓶颈”部分, 使每个功能段的时间均为 Δt 。

- 例如: 对前面的4段流水线
- 把瓶颈段 S_3 细分为3个子流水线段: S_{3a} , S_{3b} , S_{3c}

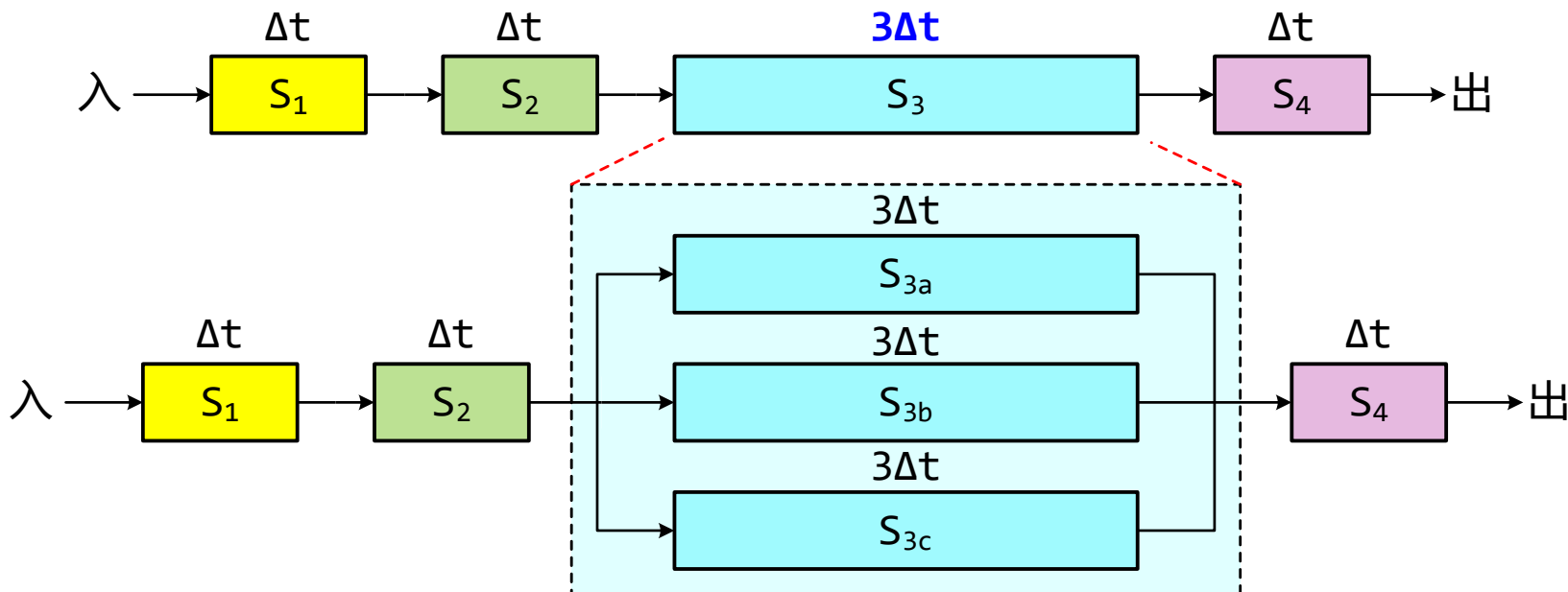


- 改进后的流水线的吞吐率: $TP_{\max} = \frac{1}{\Delta t}$

流水线“瓶颈”问题的解决方法

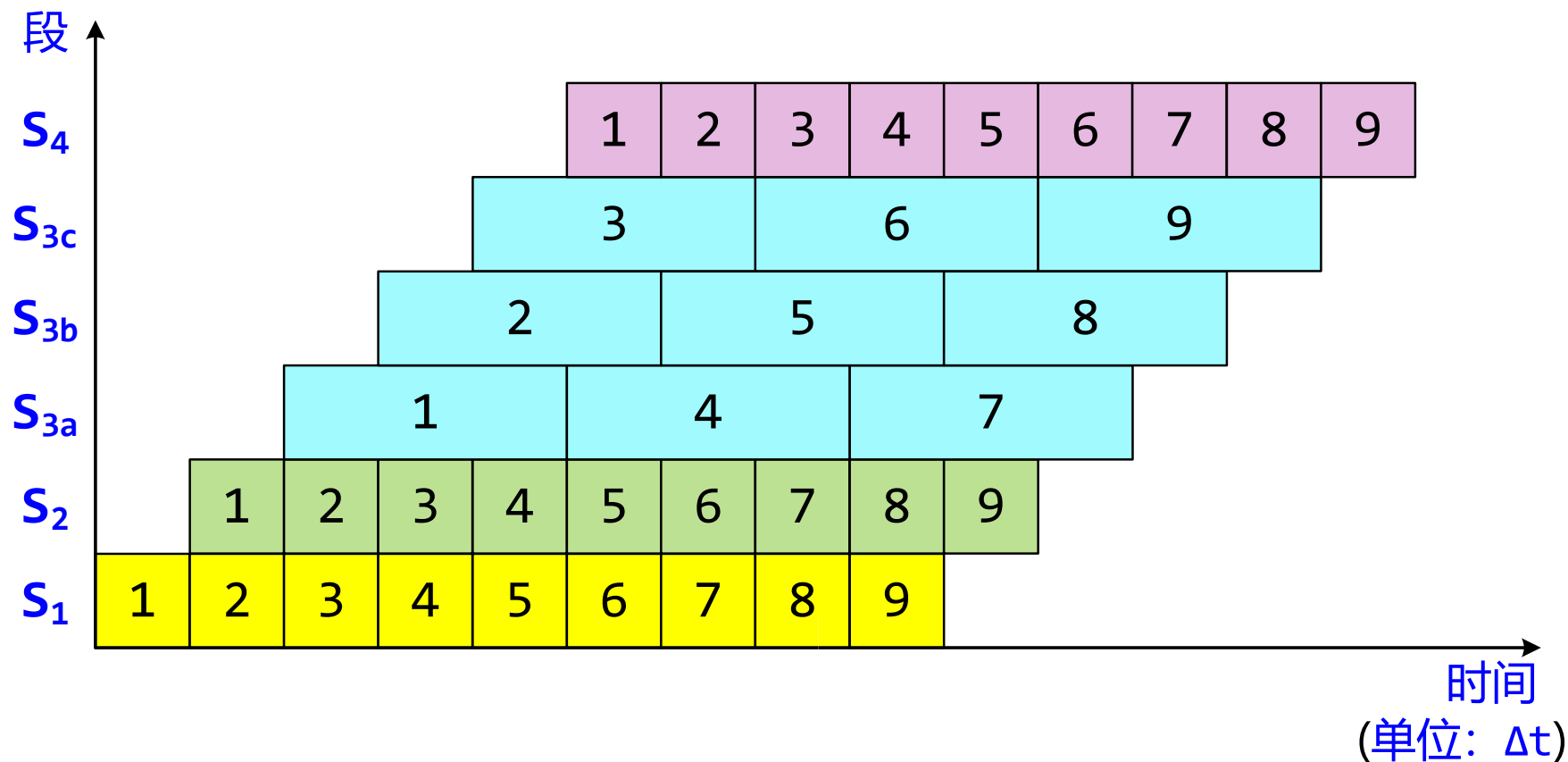
- ◆ 方法2: **重复设置“瓶颈”**，让多个瓶颈功能段并行工作。

- 例如：对前面的4段流水线
- 重复设置瓶颈段S3: S_{3a} , S_{3b} , S_{3c}



- **缺点：**控制逻辑比较复杂，所需的硬件增加了。

重复设置瓶颈段后的时空图



3.2.2 加速比

◆ 加速比：

- **定义：**完成同样一批任务，不使用流水线所用的时间与使用流水线所用的时间之比。
- 假设：不使用流水线（即顺序执行）所用的时间为 T_s ，使用流水线后所用的时间为 T_k ，则该流水线的加速比为 $S = \frac{T_s}{T_k}$

◆ 流水线各段时间相等（都是 Δt ）

- 一条 k 段流水线完成 n 个连续任务，所需要的时间为 $T_k = (k+n-1)\Delta t$
- 顺序执行 n 个任务，所需要的时间： $T_s = nk\Delta t$
- 流水线的实际加速比为

$$S = \frac{T_s}{T_k} = \frac{nk\Delta t}{(k+n-1)\Delta t} = \frac{nk}{k+n-1}$$

3.2.2 加速比

◆ 线性流水线的最大加速比：

$$S_{\max} = \lim_{n \rightarrow \infty} \frac{nk}{k + n - 1} = k$$

- 当 $n \gg k$ 时，在流水线的各段执行时间均相等的情况下，最大加速比等于流水线的段数。
- **思考**：流水线的段数愈多愈好？

◆ 流水线的各段时间不完全相等时

- 一条 k 段流水线完成 n 个连续任务的实际加速比为

$$S = \frac{n \sum_{i=1}^k \Delta t_i}{\sum_{i=1}^k \Delta t_i + (n - 1) \max(\Delta t_1, \Delta t_2, \dots, \Delta t_k)}$$

3.2.3 效率

◆ 效率：

- **定义**：流水线中的设备实际使用时间与整个运行时间的比值，即流水线设备的效率或利用率。
 - 由于流水线有通过时间和排空时间，所以在连续完成 n 个任务的时间内，各段并不是满负荷地工作。
- ### ◆ 一个 k 段线性流水线，各段执行时间相同，连续完成 n 个任务，有
- 第 i 段的实际使用时间为： $T_i = n\Delta t$
 - 整个运行时间为： $T_k = k\Delta t + (n - 1)\Delta t = (k + n - 1)\Delta t$
 - 第 i 段的效率为： $e_i = \frac{T_i}{T_k} = \frac{n\Delta t}{(k + n - 1)\Delta t} = \frac{n}{k + n - 1}$
 - 所有 k 段的效率为： $e_1 = e_2 = e_3 = \cdots = e_i = \cdots = e_k$

3.2.3 效率

- ◆ 整条流水线的效率：

$$E = \frac{e_1 + e_2 + \cdots + e_k}{k} = \frac{ke_1}{k} = \frac{kn\Delta t}{kT_k}$$

- ◆ 还可以表示为

$$E = \frac{n}{k + n - 1}$$

- ◆ 最高效率为

$$E_{\max} = \lim_{n \rightarrow \infty} \frac{n}{k + n - 1} = 1$$

- 当 $n \gg k$ 时, $E \approx 1$ 。

3.2.3 效率

- ◆ 当流水线各段时间相等时，
流水线的效率与吞吐率成正比

- 已有：
$$TP = \frac{n}{(k + n - 1)\Delta t}$$

- 可得：
$$E = TP\Delta t$$

- ◆ 流水线的效率和实际加速比S
的关系：

- 已有：
$$S = \frac{nk}{k + n - 1}$$

- 可得：
$$E = \frac{S}{k}$$

流水线的效率

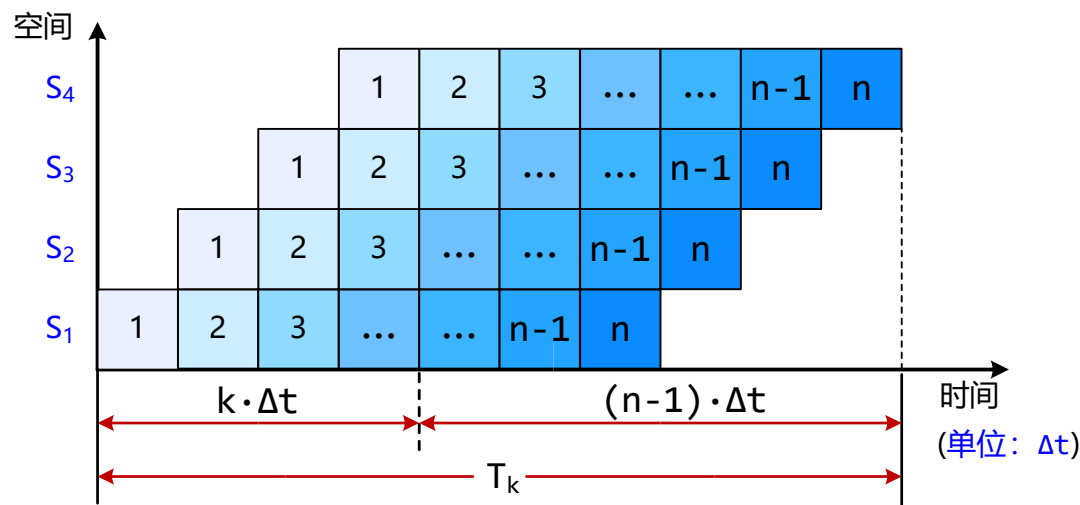
$$E = \frac{n}{k + n - 1}$$

当 $E = 1$ 时， $S = k$ ，
实际加速比达到最大

3.2.3 效率

- 已知整条流水线的效率为：

$$E = \frac{kn\Delta t}{kT_k}$$



- 从时空图上看，效率就是n个任务占用的时空面积和k个段总的时空面积之比。

$$E = \frac{n \text{ 个任务实际占用的时空区}}{k \text{ 个段总的时空区}}$$

- 当各段时间不相等时，有

$$E = \frac{n \cdot \sum_{i=1}^k \Delta t_i}{k \left[\sum_{i=1}^k \Delta t_i + (n-1) \cdot \max(\Delta t_1, \Delta t_2, \dots, \Delta t_k) \right]}$$

线性流水线最佳段数

- 流水线的段数增加，流水线的加速比和吞吐率都增加，但价格和延迟也将增加。

- 设：某任务在非流水线上的执行时间为 t ，则在 k 段流水线上的执行时间为：

- $t/k + d$,
- d 为锁存器的延迟时间

- 此时，流水线的最大吞吐率为：
$$P = \frac{1}{t/k + d}$$

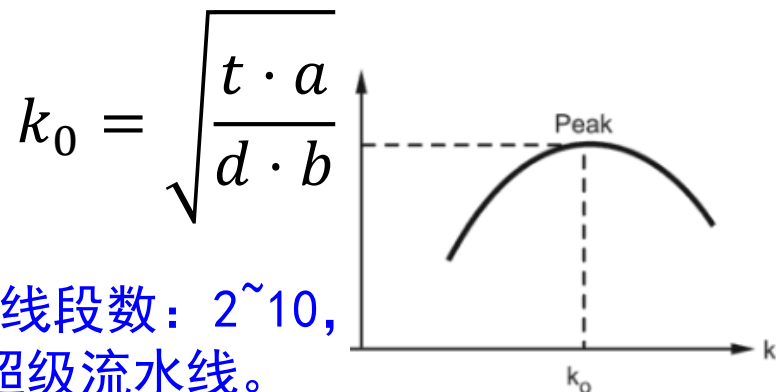
- 流水线的总价格估计为

- $C = a + bk$,
- a 为所有功能段的总价格
- b 为每个锁存器的价格

- Larson定义了PCR:

$$PCR = \frac{P}{C} = \frac{1}{t/k + d} * \frac{1}{a + bk}$$

- 流水线最佳段数



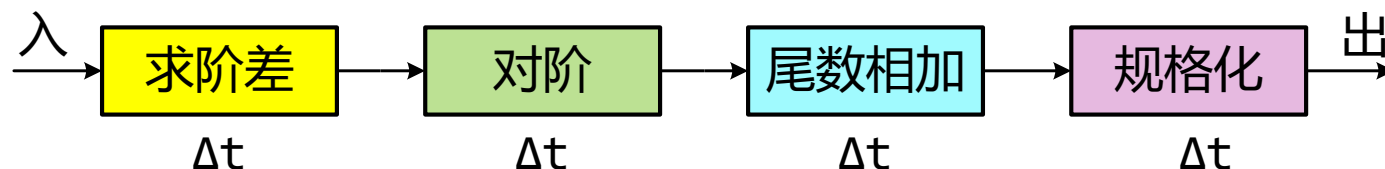
一般的流水线段数：2~10，
超过8即为超级流水线。

线性流水线性性能分析举例

◆ 实例1：单功能、线性流水线、输入任务不连续

- 用下图所示流水线计算，

$$Z = A + B + C + D + E + F + G + H$$



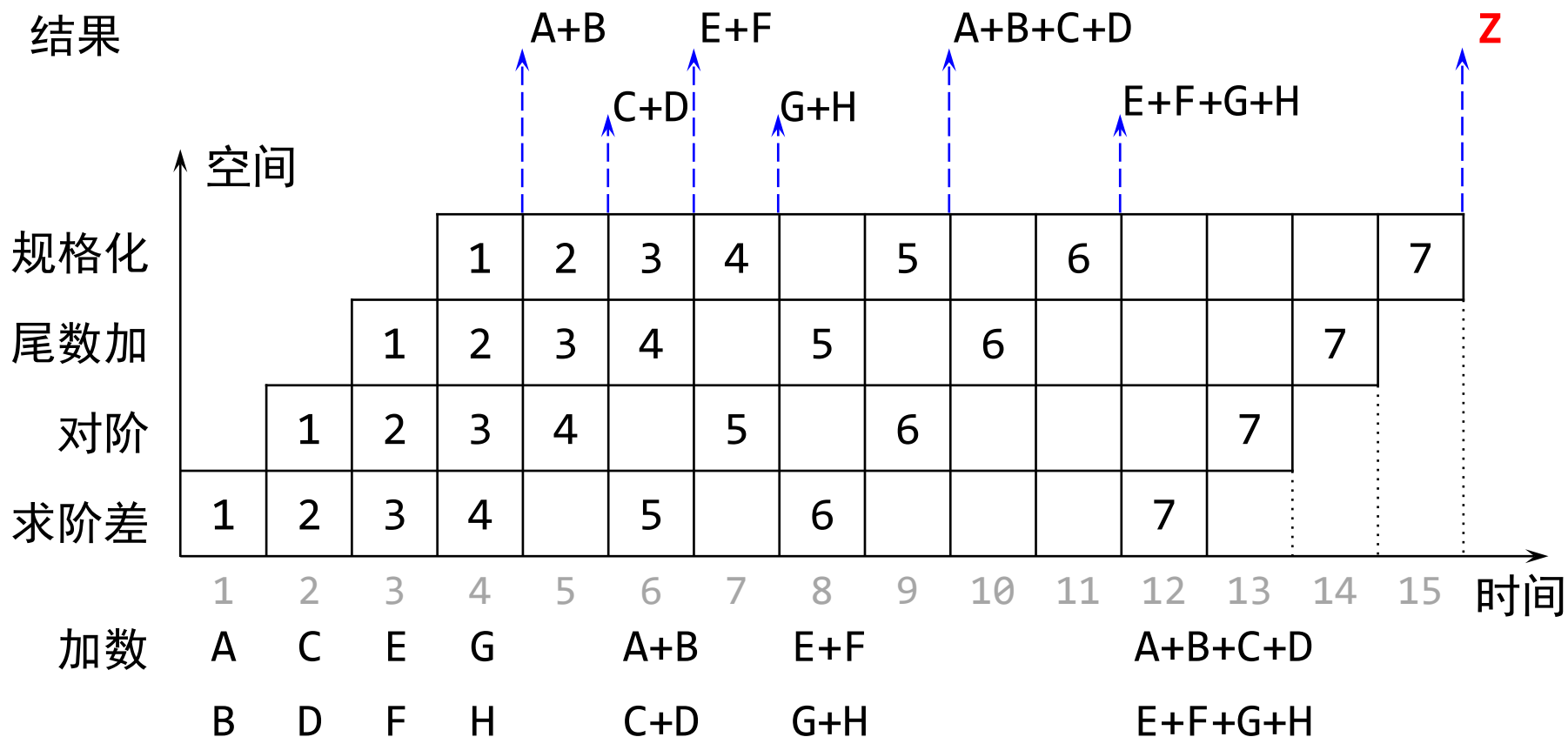
◆ 求解思路：

◆ 1) 消除数据关联

- 将 $Z = A + B + C + D + E + F + G + H$
- 转换成： $Z = [(A+B)+(C+D)] + [(E+F)+(G+H)]$

线性流水线性性能分析举例

◆ 2) 画出时空图



线性流水线性性能分析举例

◆ 3) 计算吞吐率 TP 、加速比 S 和效率 E

- 7个浮点加法共用了15个时钟周期。
- 流水线的吞吐率为：

$$TP = \frac{n}{T_k} = \frac{7}{15 \cdot \Delta t} = 0.47 \frac{1}{\Delta t}$$

$$TP_{\max} = \frac{1}{\Delta t}$$

- 流水线的加速比为：

$$S = \frac{T_0}{T_k} = \frac{4 \times 7 \cdot \Delta t}{15 \cdot \Delta t} = 1.87$$

$$S_{\max} = 4$$

- 流水线的效率为：

$$E = \frac{T_0}{k \cdot T_k} = \frac{4 \times 7 \cdot \Delta t}{4 \times 15 \cdot \Delta t} = 0.47$$

$$E_{\max} = 1$$

流水线的效率

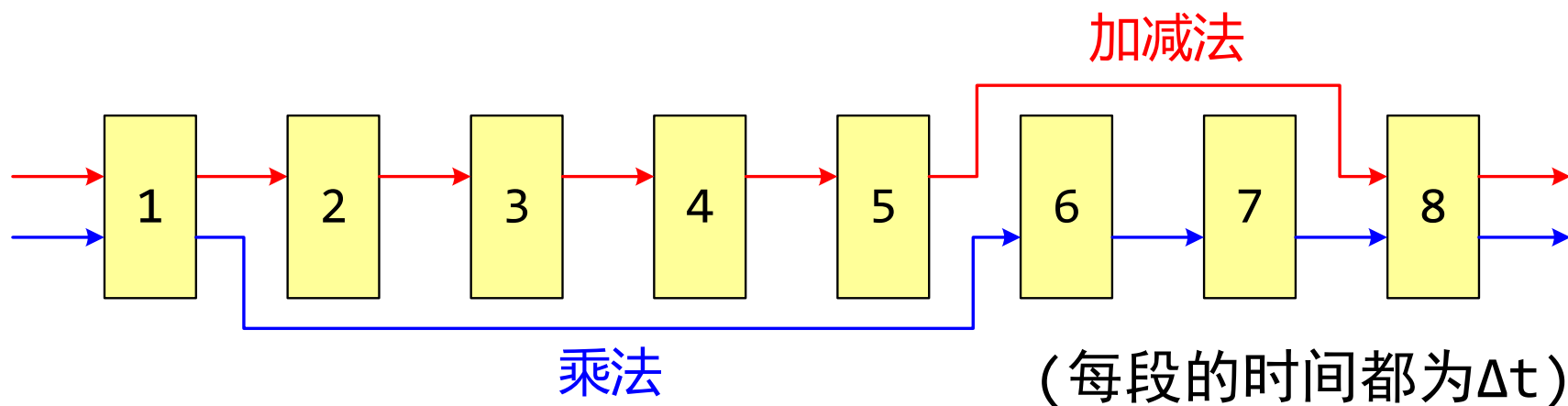
- ◆ 流水线的实际吞吐率 TP 往往达不到最大吞吐率，原因在于：
 - 各个流水线段的执行时间不等；
 - 流水线存在装入和排空时间；
 - 输入的任务一般不连续；
 - 程序存在数据相关；
 - 有的功能段不能同时使用。

多功能流水线性能分析举例

- ◆ 例3.1 设在下图所示的**静态流水线**上计算：

$$\prod_{i=1}^4 (A_i + B_i)$$

- 流水线的输出可以直接返回输入端或暂存于相应的流水寄存器中
- 试计算其吞吐率、加速比和效率。



多功能流水线性能分析举例

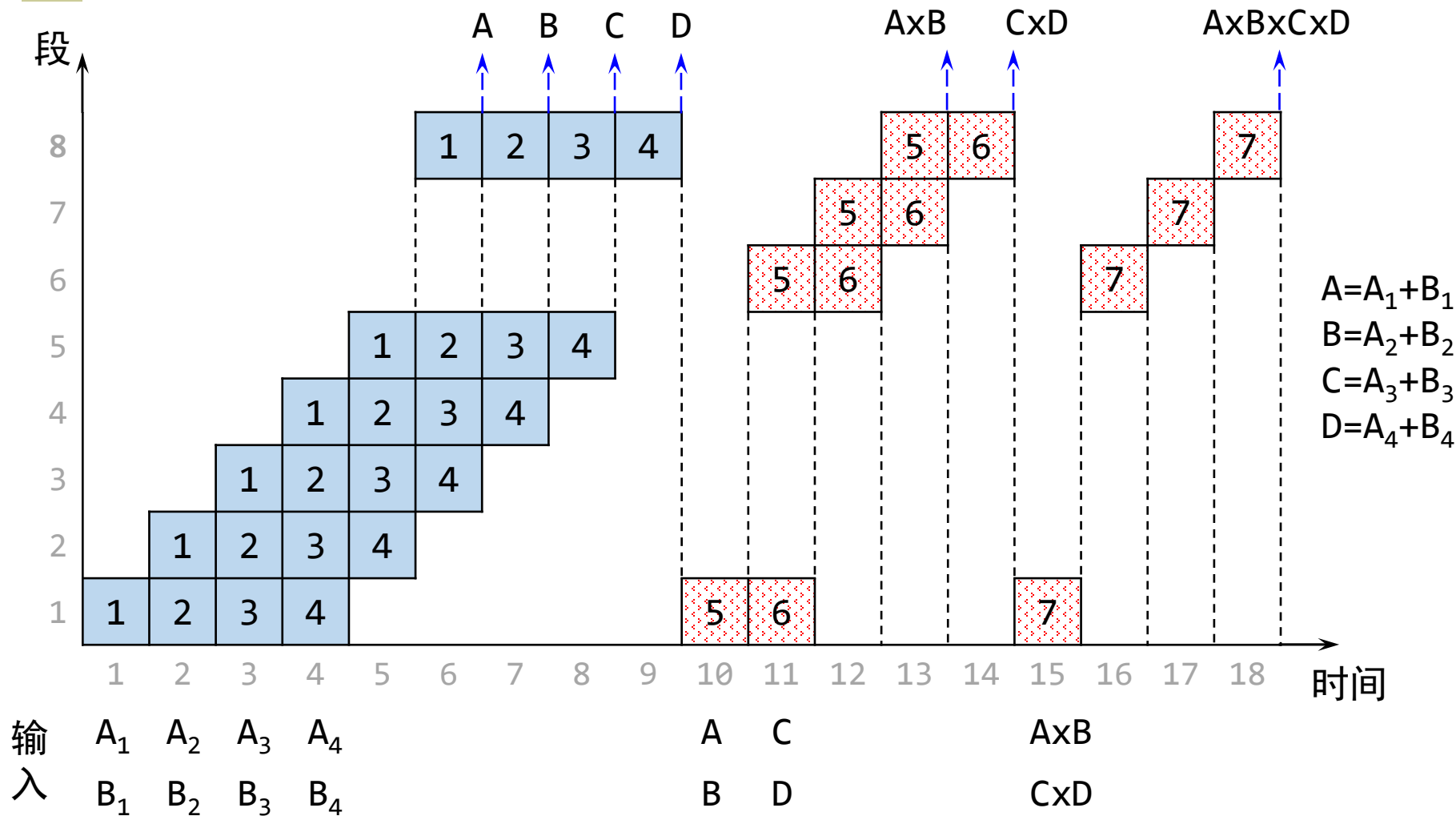
◆ 求解思路

◆ 1) 选择适合于流水线工作的算法

- 先计算 A_1+B_1 、 A_2+B_2 、 A_3+B_3 和 A_4+B_4 ;
- 再计算 $(A_1+B_1) \times (A_2+B_2)$ 和 $(A_3+B_3) \times (A_4+B_4)$;
- 然后求总的乘积结果。

◆ 2) 画出时空图

多功能流水线性能分析举例



时空图

多功能流水线性能分析举例

◆ 3) 计算性能

- **吞吐率**：在18个 Δt 时间中，给出了7个结果，有

$$TP = \frac{7}{18\Delta t}$$

- **加速比**：不用流水线，由于一次求和需 $6\Delta t$ ，一次求积需 $4\Delta t$ ，则上述7个结果共需 $(4 \times 6 + 3 \times 4)\Delta t = 36\Delta t$ ，有

$$S = \frac{36\Delta t}{18\Delta t} = 2$$

- **效率**：
$$E = \frac{4 \times 6 + 3 \times 4}{8 \times 18} = 0.25$$

- 可以看出，在求解此问题时，该流水线的效率不高。

多功能流水线性能分析举例

◆ 主要原因

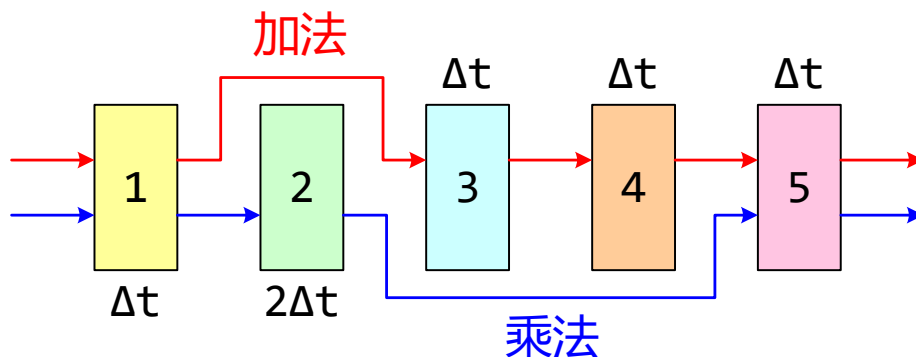
- 多功能流水线在做某一种运算时，总有一些段是空闲的。
- **静态流水线**在进行功能切换时，要等前一种运算全部流出流水线后才能进行后面的运算。
- 运算之间存在**关联**，后面有些运算要用到前面运算的结果。
- 流水线的工作过程有建立与排空部分。

多功能流水线性能分析举例

- ◆ 例3.2 有一条**动态多功能**流水线由5段组成，
 - 加法用1、3、4、5段，乘法用1、2、5段，
 - 第2段的时间为 $2\Delta t$ ，其余各段时间均为 Δt ，
 - 而且流水线的输出可以直接返回输入端或暂存于相应的流水寄存器中。若在该流水线上计算：

$$\sum_{i=1}^4 (A_i \times B_i)$$

- 试计算其吞吐率、加速比和效率。



多功能流水线性能分析举例

◆ 求解思路

◆ 1) 选择适合于流水线工作的算法

- 应先计算 $A_1 \times B_1$ 、 $A_2 \times B_2$ 、 $A_3 \times B_3$ 和 $A_4 \times B_4$;
- 再计算 $(A_1 \times B_1) + (A_2 \times B_2)$, $(A_3 \times B_3) + (A_4 \times B_4)$;
- 然后求总的累加结果。

◆ 2) 画出时空图

◆ 3) 计算性能

Figure 1 is a Gantt chart illustrating the execution of a parallel program with 7 processors. The vertical axis represents processors (1-5) and the horizontal axis represents time (1-18). Blue blocks represent tasks A, B, C, D, and their sums. Red blocks represent tasks A+B, B+C, C+D, and A+B+C+D. Dashed lines indicate task dependencies. The chart shows how tasks are scheduled across processors over time, with some processors idle at certain points.

$$TP = \frac{7}{18\Delta t}$$

$$S = \frac{28\Delta t}{18\Delta t} \approx 1.56$$

$$E = \frac{4 \times 4 + 3 \times 4}{5 \times 18} \approx 0.31$$

3.2.5 流水线设计中的若干问题

◆ 瓶颈问题

- 理想情况下，流水线在工作时，其中的任务是同步地每一个时钟周期往前流动一段。
- 当流水线各段不均匀时，机器的时钟周期取决于瓶颈段的延迟时间。
- 在设计流水线时，要尽可能使各段时间相等。

◆ 流水线的额外开销

- 流水寄存器延迟
 - 流水寄存器需要建立时间和传输延迟
- 时钟偏移开销
 - 流水线中，时钟到达各流水寄存器的最大差值时间

3.2.5 流水线设计中的若干问题

◆ 几个问题

- 流水线并不能减少（而且一般是增加）单条指令的执行时间，但却能提高吞吐率。
- 增加流水线的深度（段数）可以提高流水线的性能。
- 流水线的深度受限于流水线的额外开销。
- 当时钟周期小到与额外开销相同时，流水已没意义。因为这时在每一个时钟周期中已没有时间来做有用的工作。

◆ 冲突问题

- 流水线设计中要解决的重要问题之一。

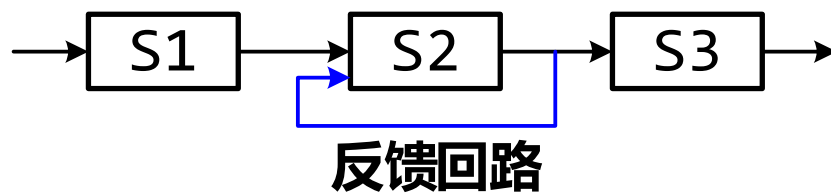
3.3 非线性流水线的调度

- ◆ 在非线性流水线中，存在反馈回路，当一个任务在流水线中流过时，可能要多次经过某些段。
- ◆ 要解决的问题：
 - 应按什么样的时间间隔向流水线输入新任务，才能既不发生功能段使用冲突，又能使流水线有较高的吞吐率和效率？
- ◆ 3.3.1 **单功能**非线性流水线的最优调度
- ◆ 3.3.2 多功能非线性流水线的调度
- ◆ 向一条非线性流水线的输入端连续输入两个任务之间的时间间隔称为非线性流水线的**启动距离**。
- ◆ 会引起非线性流水线功能段使用冲突的启动距离则称为**禁用启动距离**。

3.3.1 单功能非线性流水线的最优调度

◆ 流水线预约表：

- 反映流水线各功能段的执行时序。横坐标为时间（时钟周期数），纵坐标为流水线功能段。



		时间 →			
		1	2	3	4
功能段 ↓	S1	✓			
	S2		✓	✓	
	S3				✓

◆ 说明

- 如果在第 n 个时钟周期使用第 k 段，则在第 k 行和第 n 列的交叉处的格子里有一个 ✓

3.3.1 单功能非线性流水线的最优调度

◆ 非线性流水线的调度问题

- 确定什么时候向流水线引进新的任务，才能使该任务不会与先前进入流水线的任务发生冲突——争用流水段。

◆ 步骤

- (1) 根据预约表写出禁止表F
- (2) 根据禁止表F写出初始冲突向量 C_0
- (3) 根据初始冲突向量 C_0 画出状态转换图
- (4) 根据状态转换图写出最优调度方案

◆ (1) 根据预约表写出禁止表F

- **禁止表F**：一个由禁用启动距离构成的集合。
- 具体方法：对于预约表的每一行的任何一对 $\checkmark$ ，用它们所在的列号相减（大的减小的），列出各种可能的差值，然后删除相同的，剩下的就是禁止表的元素。

3.3.1 单功能非线性流水线的最优调度

- ◆ 以一个5功能段非线性流水线预约表为例进行说明

功能段 \ 时间	1	2	3	4	5	6	7	8	9
S1	✓								✓
S2		✓	✓					✓	
S3				✓					
S4					✓	✓			
S5							✓	✓	

- 第一行的差值只有一个：8；
 - 第二行的差值有3个：1，5，6；
 - 第3行只有一个✓，没有差值；
 - 第4和第5行的差值都只有一个：1；
- ◆ 其禁止表是： $F = \{ 1, 5, 6, 8 \}$

3.3.1 单功能非线性流水线的最优调度

◆ (2) 根据禁止表F写出初始冲突向量 C_0 。

- 完成从一个集合到一个二进制位串的变换
- 冲突向量C：一个N位的二进制位串。
- 设 $C_0 = (C_N C_{N-1} \dots C_i \dots C_2 C_1)$ ，则：

$$C_i = \begin{cases} 1 & i \in F \\ 0 & i \notin F \end{cases}$$

- $C_i=0$ ：允许间隔*i*个时钟周期后送入后续任务
- $C_i=1$ ：**不允许**间隔*i*个时钟周期后送入后续任务

◆ 对于上述的例子，有

- $F = \{ 1, 5, 6, 8 \}$
- $C_0 = (10110001)$

3.3.1 单功能非线性流水线的最优调度

◆ (3) 根据初始冲突向量 C_0 画出状态转换图

- 当第一个任务流入流水线后，初始冲突向量 C_0 决定了下一个任务需间隔多少个时钟周期才可以流入。
- 在第二个任务流入后，新的冲突向量是怎样的呢？
 - 假设第二个任务是在与第一个任务间隔 j 个时钟周期流入，这时，由于第一个任务已经在流水线中前进了 j 个时钟周期，其相应的禁止表中各元素的值都应该减去 j ，并丢弃小于等于0的值。
 - 对冲突向量来说，就是逻辑右移 j 位（左边补0）。
 - 在冲突向量上，就是对它们的冲突向量进行“或”运算

$$\text{SHR}^{(j)}(C_0) \vee C_0$$

- ◆ 其中： $\text{SHR}^{(j)}$ 表示逻辑右移 j 位

3.3.1 单功能非线性流水线的最优调度

◆ (3) 根据初始冲突向量 C_0 画出状态转换图

- 推广到更一般的情况
 - 假设： C_k ：当前的冲突向量， j ：允许的时间间隔
 - 则新的冲突向量为： $SHR^{(j)}(C_k) \vee C_0$
- 从初始冲突向量 C_0 出发，反复应用上述步骤，可以求得所有的冲突向量以及产生这些向量所对应的时间间隔。由此可以画出用冲突向量表示的流水线状态转移图。
 - 有向弧：表示状态转移的方向
 - 弧上的数字：表示引入后续任务（从而产生新的冲突向量）所用的时间间隔（时钟周期数）

3.3.1 单功能非线性流水线的最优调度

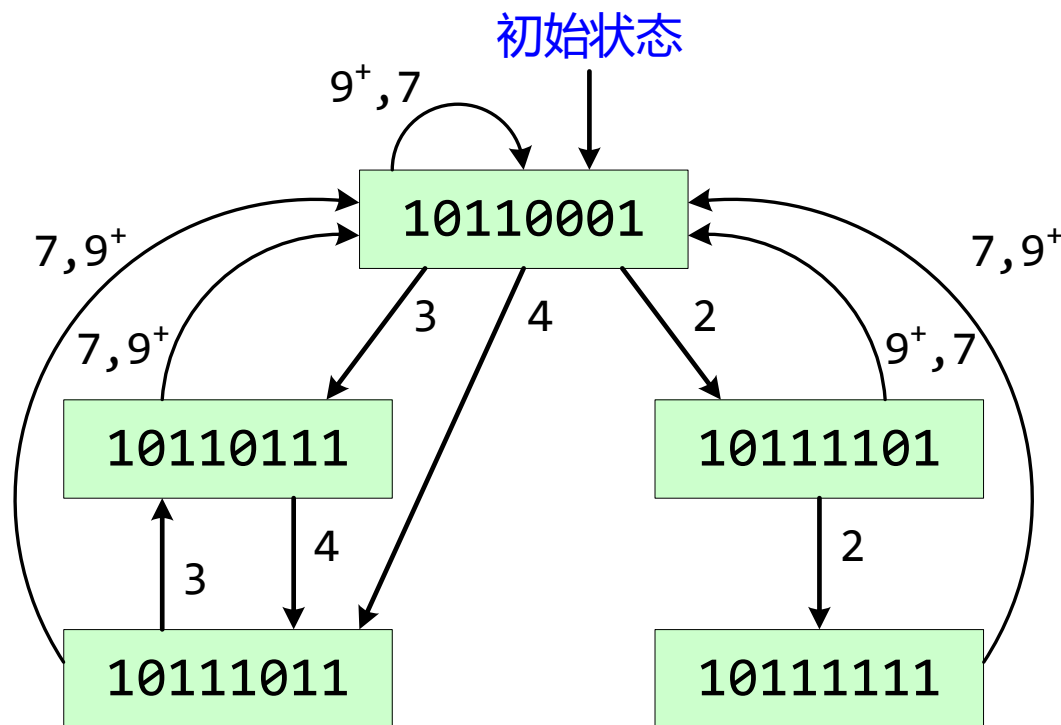
◆ (3) 根据初始冲突向量 C_0 画出状态转换图

- $C_0 = (10110001)$
- 引入后续任务可用的时间间隔为：2、3、4、7个时钟周期
- 如果采用2，则新的冲突向量为：
 $(00101100) \vee (10110001) = (10111101)$
- 如果采用3，则新的冲突向量为：
 $(00010110) \vee (10110001) = (10110111)$
- 如果采用4，则新的冲突向量为：
 $(00001011) \vee (10110001) = (10111011)$
- 如果采用7，则新的冲突向量为：
 $(00000001) \vee (10110001) = (10110001)$

3.3.1 单功能非线性流水线的最优调度

◆ (3) 根据初始冲突向量 C_0 画出状态转换图

- 对于新的向量 $C_1 = (10111101)$
- 引其可用的时间间隔为2个和7个时钟周期。用类似上面的方法，可以求出其后续的冲突向量分别为 (10111101) 和 (10110001) 。
- 对于其他新向量，也照此处理。
- 在此基础上，画出状态转移示意图。



3.3.1 单功能非线性流水线的最优调度

◆ (4) 根据状态转换图写出最优调度方案

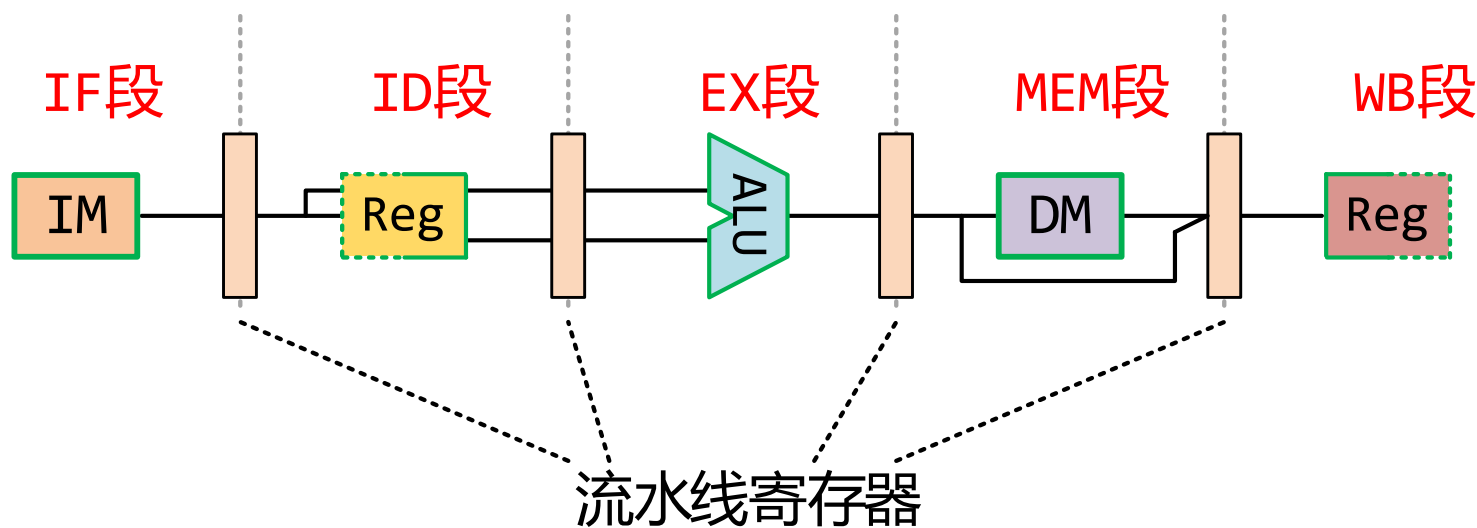
- 根据流水线状态图，由初始状态出发，任何一个**闭合回路**即为一种调度方案。
- 列出所有可能的调度方案，计算出每种方案的平均时间间隔，从中找出其最小者即为最优调度方案。
- 上例中的最佳方案
 - (3, 4)，平均间隔时间3.5个时钟周期（吞吐率最高）
 - 方案(4, 3)的平均间隔时间也是3.5，是否也是最佳方案，为什么？
 - 缺点：方案(3, 4)是一种不等时间间隔的调度方案，与等间隔的调度方案相比，在控制上要复杂得多。
 - 等时间间隔的方案只有一个：(7)，其吞吐率下降了一半

3.4 流水线的相关与冲突

- ◆ 一条指令的执行过程分为以下5个周期：
 - 取指令周期（IF）
 - 指令译码/读寄存器周期（ID）
 - 执行/有效地址计算周期（EX）
 - 存储器访问 / 分支完成周期（MEM）
 - 写回周期（WB）
- ◆ 在这个实现方案中：
 - 分支指令需要4个时钟周期（如果把分支指令的执行提前到ID周期，则只需要2个周期）。
 - store指令需要4个周期。
 - 其它指令需要5个周期才能完成。

一个经典的5段流水线

- ◆ 将上述实现方案修改为流水线实现
 - 每一个周期作为一个流水段。
 - 在各段之间加上锁存器（流水寄存器）。

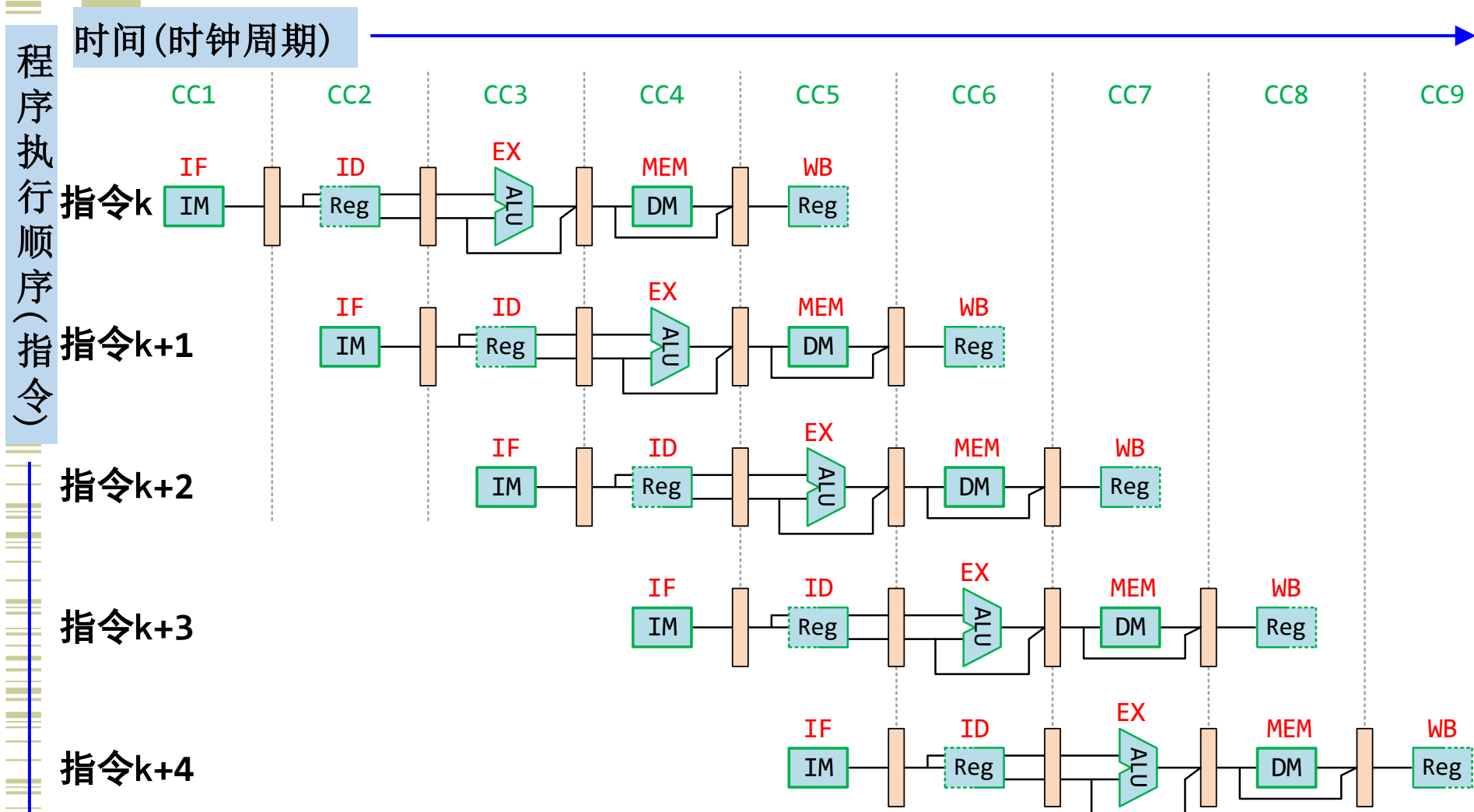


一个经典的5段流水线

时间周期 指令	1	2	3	4	5	6	7	8	9
指令k	IF	ID	EX	MEM	WB				
指令k+1		IF	ID	EX	MEM	WB			
指令k+2			IF	ID	EX	MEM	WB		
指令k+3				IF	ID	EX	MEM	WB	
指令k+4					IF	ID	EX	MEM	WB

5段流水线的另一种时空图

一个经典的5段流水线



5段流水线按时间错开的数据通路序列

3.4 流水线的相关与冲突

- ◆ 采用流水线方式实现时，应解决以下问题：
 - 要保证不会在同一时钟周期要求同一个功能段做两件不同的工作。
 - 例如，不能要求ALU同时做有效地址计算和算术运算。
 - 避免**IF段的访存**（取指令）与**MEM段的访存**（读/写数据）发生冲突。
 - 可以采用分离的指令存储器和数据存储器；
 - 一般采用分离的指令Cache和数据Cache。
 - ID段和WB段都要访问同一寄存器文件。
 - ID段：读 WB段：写
 - **如何解决**？把写操作安排在时钟周期的**前半拍**完成，把读操作安排在**后半拍**完成。

3.4 流水线的相关与冲突

◆ 考虑PC的问题

- 流水线为了能够每个时钟周期启动一条新的指令，就必须在每个时钟周期进行PC值的加4操作，并保留新的PC值。
 - 这种操作必须在IF段完成，以便为取下一条指令做好准备。
 - 需设置一个专门的加法器
- 但分支指令也可能改变PC的值，而且是在MEM段进行，这会导致冲突。
 - **请考虑一下，如何处理分支指令？**

3.4.2 相关与流水线冲突

- ◆ **相关**：两条指令之间存在某种依赖关系。
 - 如果两条指令相关，则它们就有可能不能在流水线中重叠执行或者只能部分重叠执行。
- ◆ 相关有3种类型
 - 数据相关（也称真数据相关），指令i（前）和j（后）：
 - 指令j使用指令i产生的结果；
 - 指令j与指令k数据相关，而指令k又与指令i数据相关。
 - 名相关
 - 反相关：如果指令j写的名与指令i读的名相同
 - 输出相关：如果指令j和指令i写相同的名
 - 控制相关
 - 由分支指令引起的相关
 - 为保证程序应有的执行顺序，必须严格按控制相关确定的顺序执行

换名技术

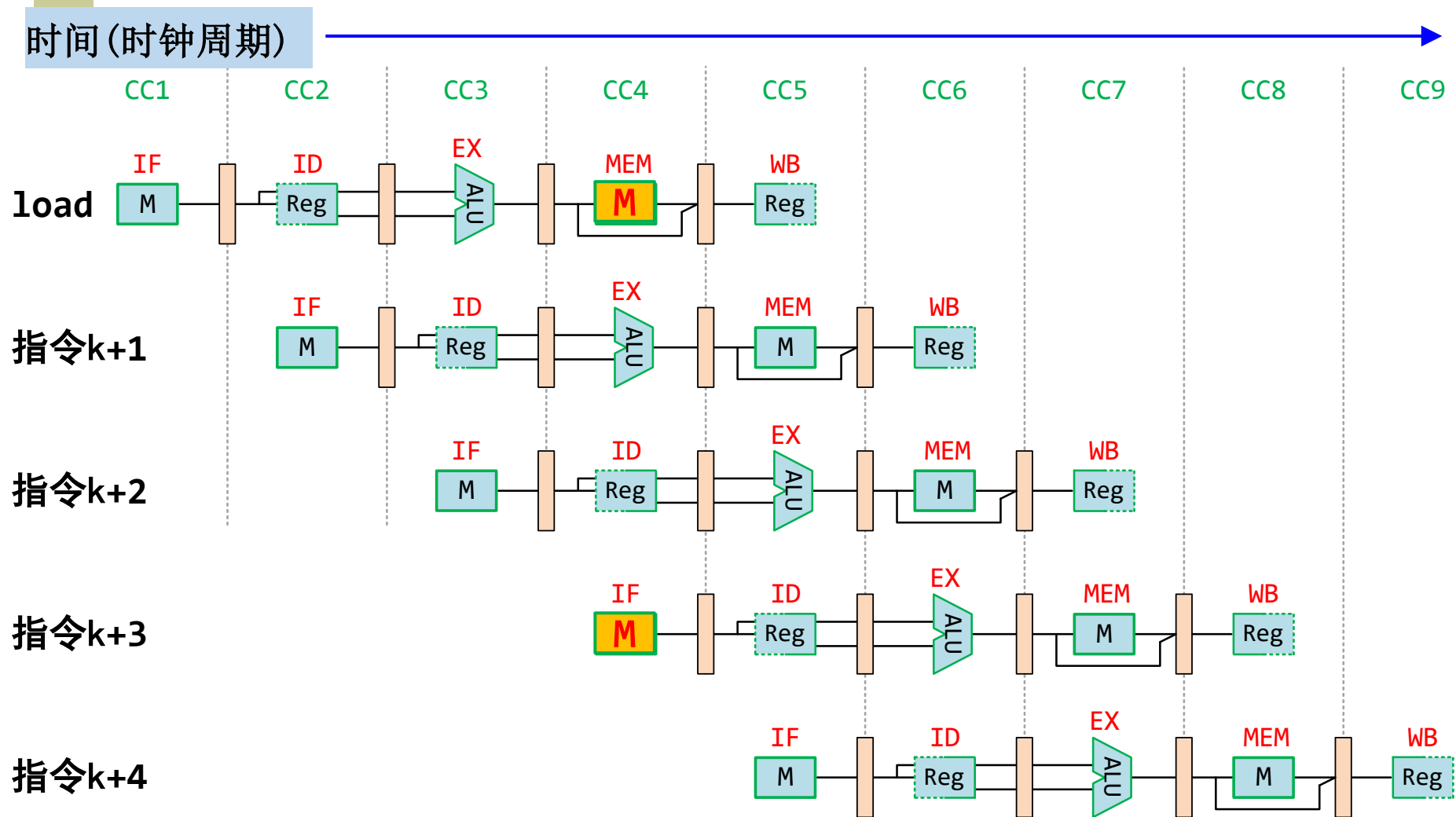
3.4.2 相关与流水线冲突

- ◆ **流水线冲突**：指对于具体的流水线来说，由于相关的存在，使得指令流中的下一条指令不能在指定的时钟周期执行。
 - **结构冲突**：因硬件资源满足不了指令重叠执行的要求而发生的冲突。
 - **数据冲突**：当指令在流水线中重叠执行时，因需要用到前面指令的执行结果而发生的冲突。
 - **控制冲突**：流水线遇到分支指令和其他会改变PC值的指令所引起的冲突。
- ◆ **流水线冲突带来的问题**
 - 导致错误的执行结果。
 - 流水线可能会出现停顿，降低流水线的效率和实际的加速比。
 - 当一条指令被暂停时，在该暂停指令之后流出的所有指令都要被暂停，之前流出的指令则继续进行

流水线冲突—结构冲突

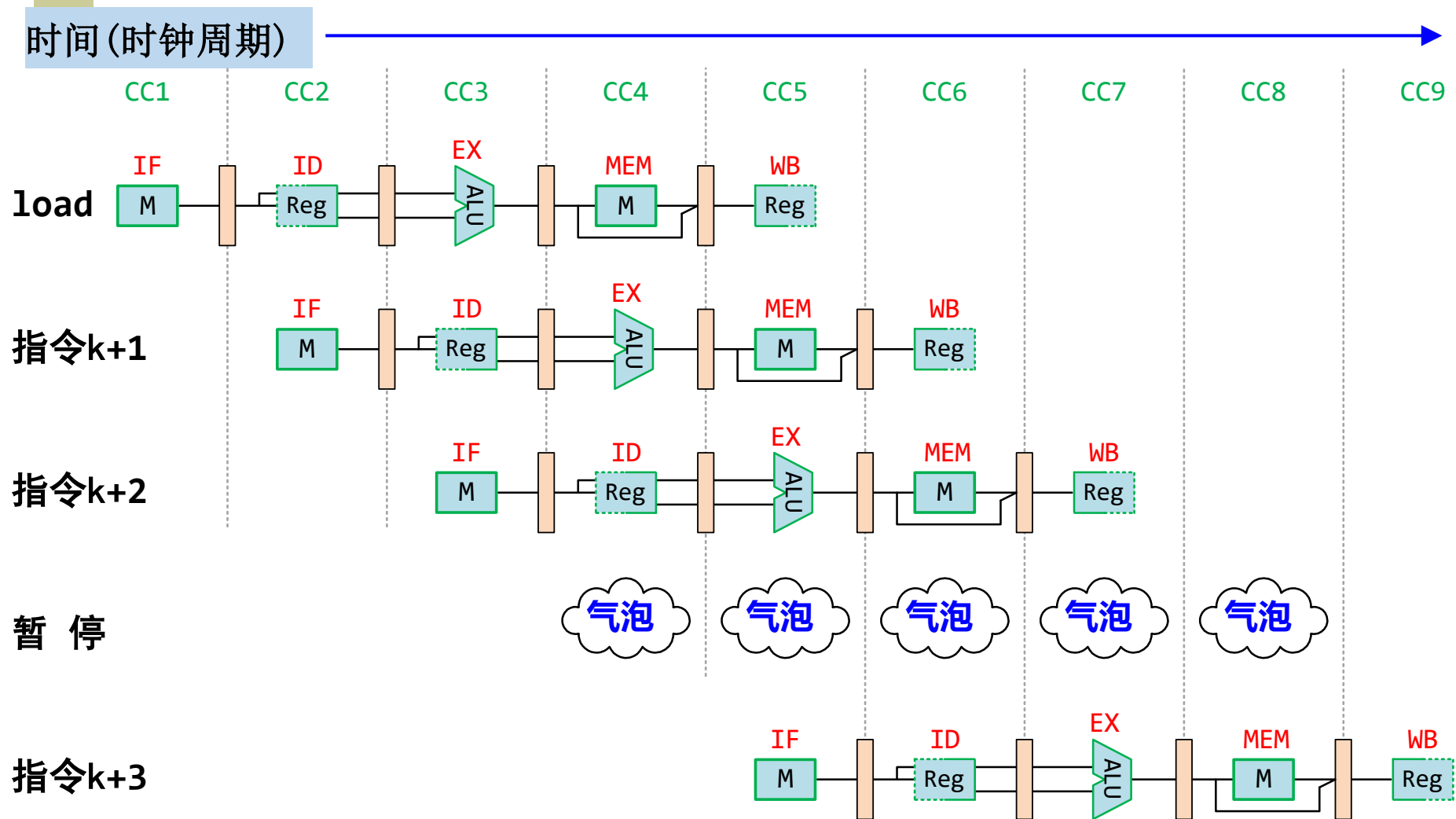
- ◆ **结构冲突**：如果某种指令组合因为资源冲突而不能正常执行，则称该处理机有结构冲突。
- ◆ 常见的导致结构相关的原因：
 - 功能部件不是完全流水
 - 资源份数不够
- ◆ 结构冲突举例：访存冲突
 - 有些流水线处理机**只有一个存储器**，将数据和指令放在一起，**访存指令会导致访存冲突**。
 - 解决办法：
 - 1) 插入暂停周期，“流水线气泡”
 - 2) 设置相互独立的指令存储器和数据存储器或设置相互独立的指令Cache和数据Cache。

结构冲突举例



由于访问同一个存储器而引起的结构冲突

结构冲突举例



为消除结构冲突而插入的流水线气泡

结构冲突举例

◆ 引入暂停后的时空图

指令编号	时钟周期									
	1	2	3	4	5	6	7	8	9	10
指令k	IF	ID	EX	MEM	WB					
指令k+1		IF	ID	EX	MEM	WB				
指令k+2			IF	ID	EX	MEM	WB			
指令k+3				stall	IF	ID	EX	MEM	WB	
指令k+4						IF	ID	EX	MEM	WB
指令k+5							IF	ID	EX	MEM

◆ 有时流水线设计者允许结构冲突的存在

- 主要原因：减少硬件成本

流水线冲突—数据冲突

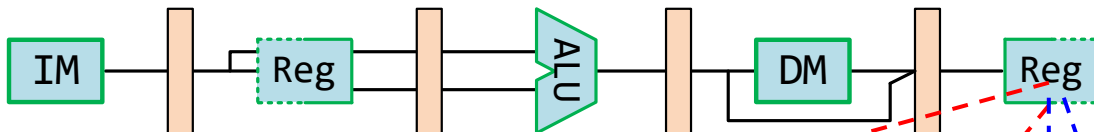
- ◆ **数据冲突**：当相关的指令靠得足够近时，它们在流水线中的重叠执行或者重新排序会改变指令读/写操作数的顺序，使之不同于它们非流水实现时的顺序，则发生了数据冲突。
- ◆ 根据指令读访问和写访问的顺序，可以将数据冲突分为3种类型。
 - 写后读冲突（RAW）
 - 在i写入之前，j先去读。j读出的内容是错误的。
 - 读后写冲突（WAR）【**流水线中写操作提前、读操作滞后**】
 - 在i读之前，j先写。i读出的内容是错误的！
 - 写后写冲突（WAW）【**流水线中多个段可以进行写操作**】
 - 在i写入之前，j先写。最后写入的结果是i的。错误！

数据相关举例

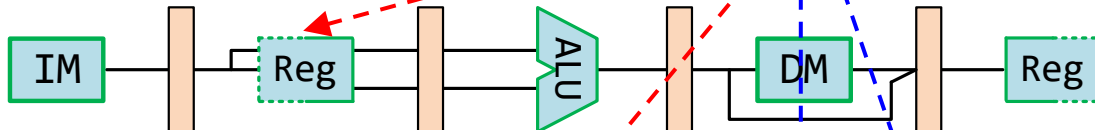
时间(时钟周期)

CC1 CC2 CC3 CC4 CC5 CC6

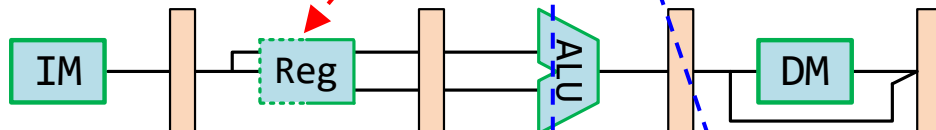
DADD R1, R2, R3



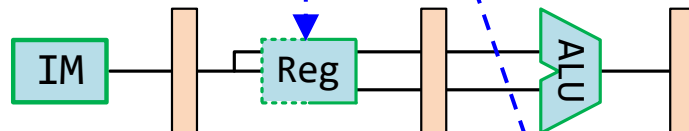
DSUB R4, **R1**, R5



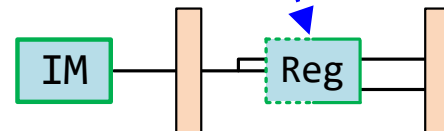
XOR R6, **R1**, R7



AND R8, **R1**, R9

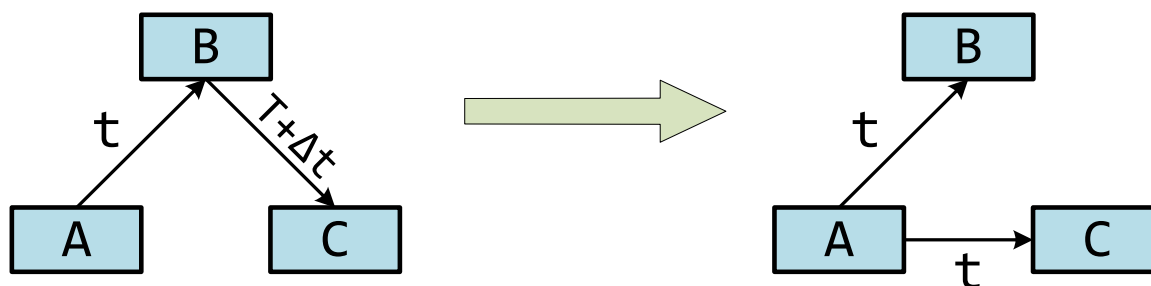


OR R10, **R1**, R11



流水线冲突—数据冲突

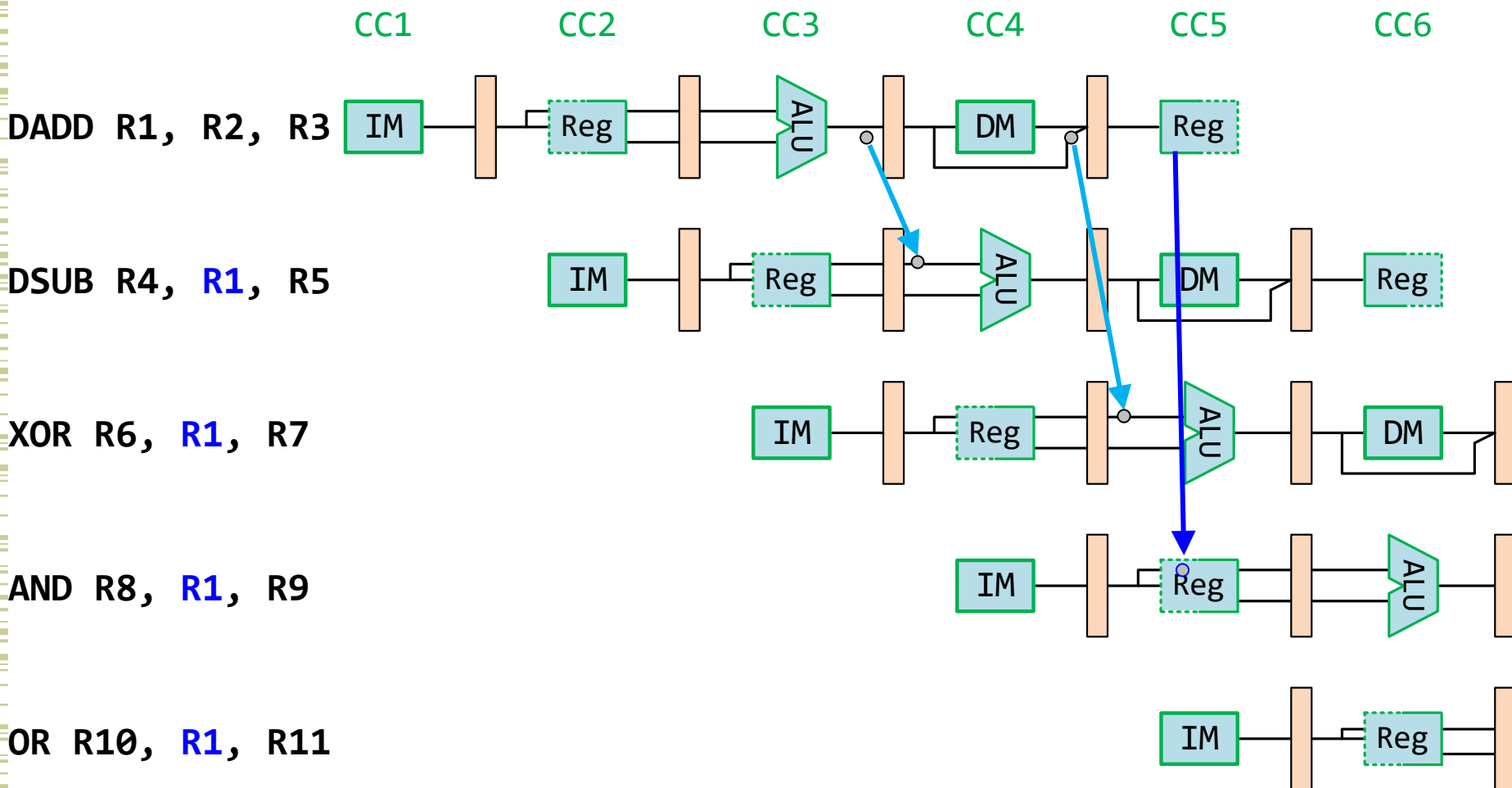
- ◆ 通过定向技术减少数据冲突引起的停顿
 - 定向技术也称为旁路或短路
- ◆ 关键思想：
 - 在某条指令产生计算结果之前，其他指令并不真正立即需要该计算结果，如果能够将该计算结果从其产生的地方直接送到其他指令需要它的地方，那么就可以避免停顿。



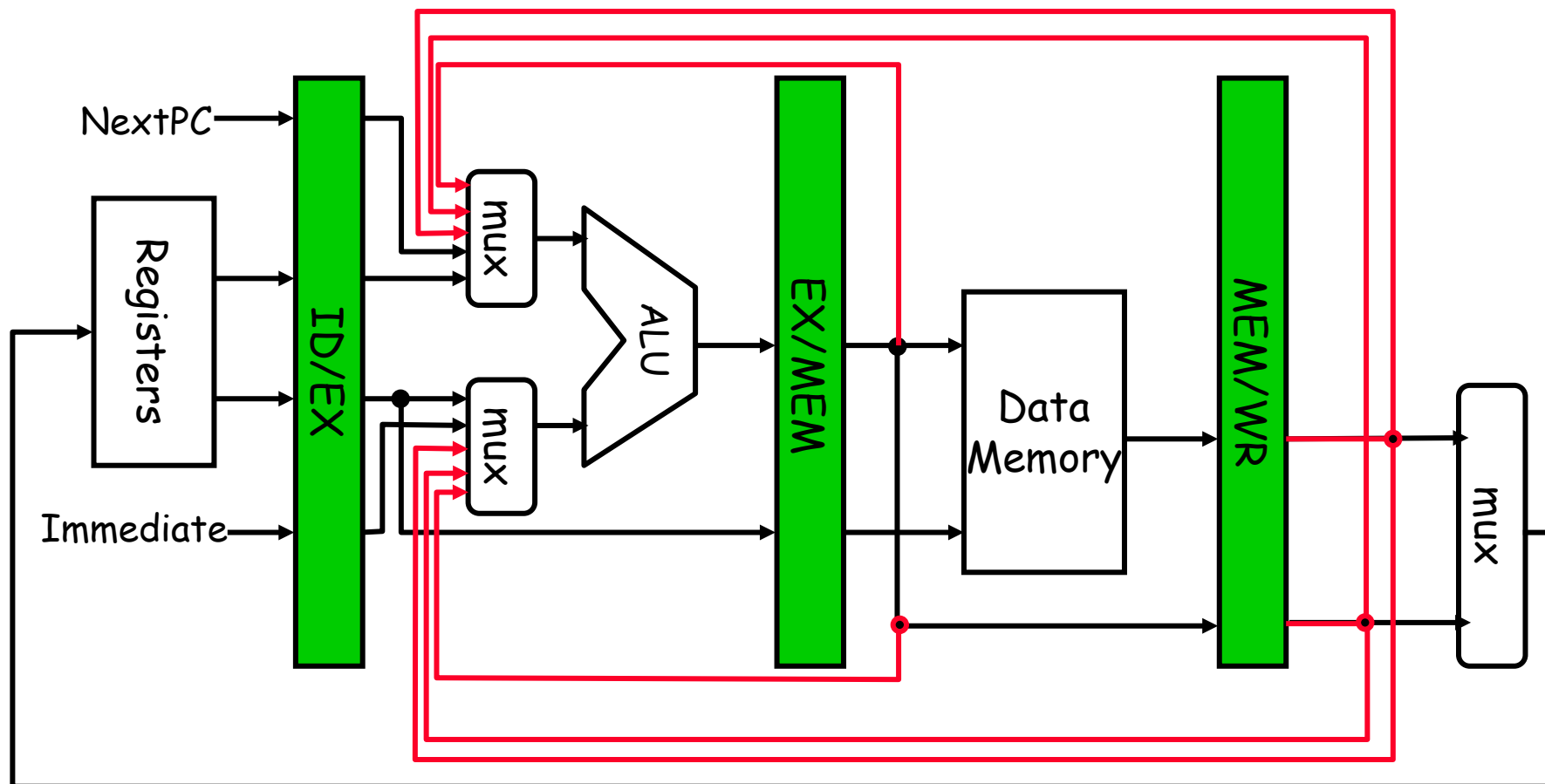
RAW相关的数据重定向

数据相关举例-定向路径

时间(时钟周期)



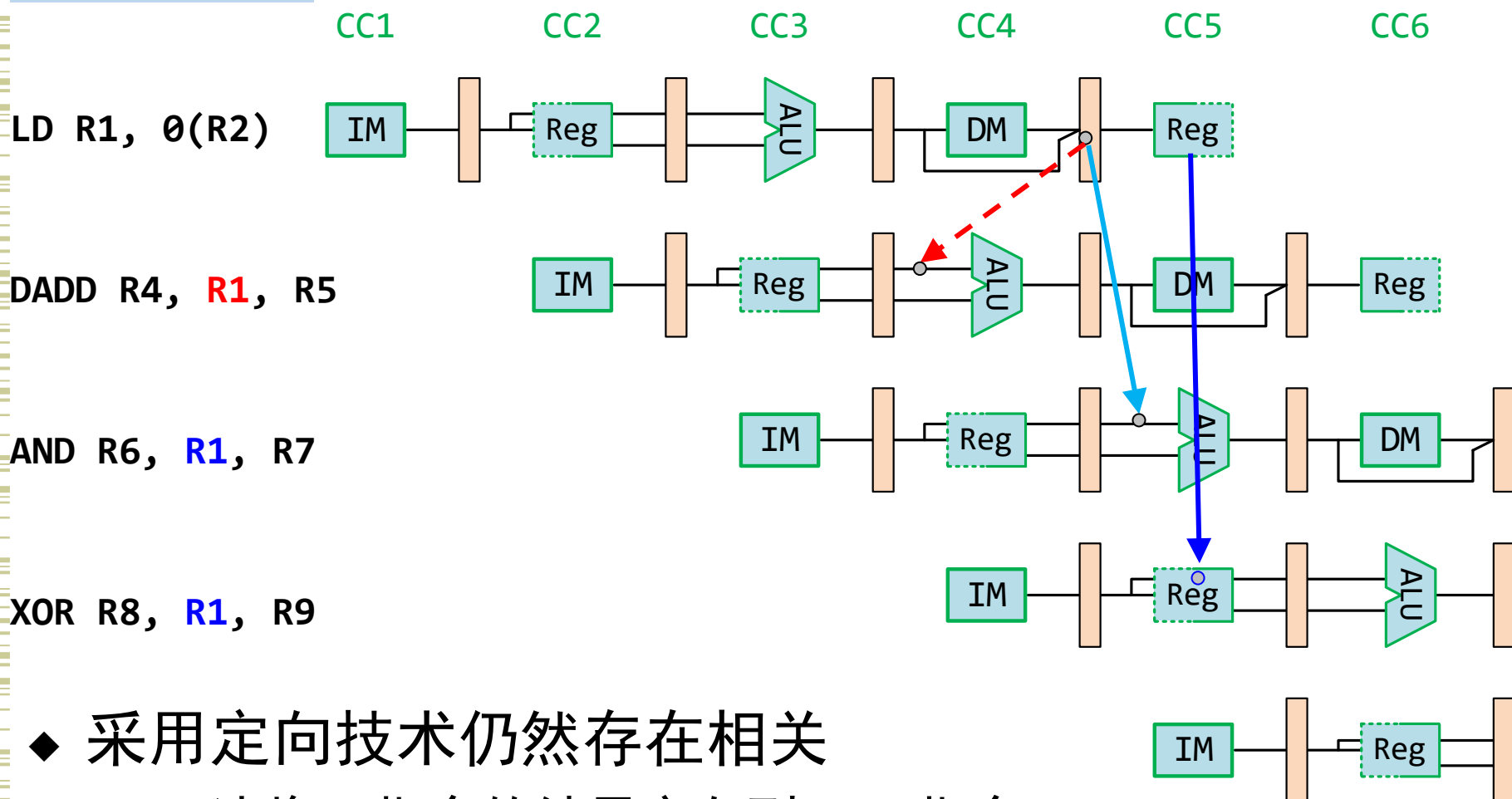
采用定向技术硬件所需做的修改



- ◆ 当定向硬件检测到前面某条指令的结果寄存器就是当前指令的源寄存器时，控制逻辑会将前面那条指令的结果直接从其产生的地方定向到当前指令所需的位置。

数据相关举例-定向路径

时间(时钟周期)



- ◆ 采用定向技术仍然存在相关
 - 无法将LD指令的结果定向到DADD指令

数据相关举例-定向路径

时间(时钟周期)

CC1

CC2

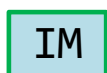
CC3

CC4

CC5

CC6

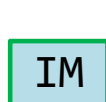
LD R1, 0(R2)



DADD R4, R1, R5



AND R6, R1, R7



XOR R8, R1, R9



- ◆ 流水线互锁机制插入气泡后的执行过程

数据相关举例-定向路径

插入停顿前

LD R1, 0 (R2)	IF	ID	EX	MEM	WB			
DADD R4, R1, R5		IF	ID	EX	MEM	WB		
AND R6, R1, R7			IF	ID	EX	MEM	WB	
XOR R8, R1, R9				IF	ID	EX	MEM	WB

插入停顿后

LD R1, 0 (R2)	IF	ID	EX	MEM	WB			
DADD R4, R1, R5		IF	ID	stall	EX	MEM	WB	
AND R6, R1, R7			IF	stall	ID	EX	MEM	WB
XOR R8, R1, R9				stall	IF	ID	EX	MEM

◆ 插入停顿前后的流水线时空图

数据相关举例-依靠编译器

- ◆ 依靠编译器解决数据冲突
 - 让编译器重新组织指令顺序来消除冲突，这种技术称为指令调度或流水线调度。
- ◆ 例如：采用典型的代码生成方法，
 - 表达式 $A=B+C$ 的代码会导致暂停

LD Rb, B	IF	ID	EX	MEM	WB				
LD Rc, C		IF	ID	EX	MEM	WB			
DADD Ra, Rb, Rc			IF	ID	stall	EX	MEM	WB	
SD Ra, A				IF	stall	ID	EX	MEM	WB

数据相关举例-依靠编译器

◆ 请为下列表达式生成没有暂停的指令序列：

- 假设载入延迟为1个时钟周期。

$A = B + C$;

$D = E - F$;

调度前的代码

LD	Rb, B
LD	Rc, C
DADD	Ra, Rb, Rc
SD	Ra, A
LD	Re, E
LD	Rf, F
DSUB	Rd, Re, Rf
SD	Rd, D

调度后的代码

LD	Rb, B
LD	Rc, C
LD	Re, E
DADD	Ra, Rb, Rc
LD	Rf, F
SD	Ra, A
DSUB	Rd, Re, Rf
SD	Rd, D

控制冲突

- ◆ 执行分支指令的结果有两种
 - 分支成功：PC值改变为分支转移的目标地址。
 - 在条件判定和转移地址计算都完成后，才改变PC值。
 - 不成功或者失败：PC的值保持正常递增，
 - 指向顺序的下一条指令。
- ◆ 处理分支指令最简单的方法：
 - “冻结” 或者 “排空” 流水线 。
 - 优点：简单。
 - 前述5段流水线中，改变PC值是在MEM段进行的。
 - 给流水线带来了3个时钟周期的延迟。

控制冲突-简单处理分支指令

◆ 简单处理分支指令：分支成功的情况

分支指令	IF	ID	EX	MEM	WB					
分支目标指令		IF	stall	stall	IF	ID	EX	MEM	WB	
分支目标指令+1						IF	ID	EX	MEM	WB
分支目标指令+2							IF	ID	EX	MEM
分支目标指令+3								IF	ID	EX

◆ 简单处理分支指令：分支失败的情况

分支指令	IF	ID	EX	MEM	WB					
分支后继指令		IF	stall	stall	IF	ID	EX	MEM	WB	
分支后继指令+1						IF	ID	EX	MEM	WB
分支后继指令+2							IF	ID	EX	MEM
分支后继指令+3								IF	ID	EX

控制冲突-简单处理分支指令

- ◆ 把由分支指令引起的延迟称为**分支延迟**。
- ◆ 分支指令在目标代码中出现的频度
 - 每3~4条指令就有一条是分支指令。
- ◆ 假设：分支指令出现的频度是30%，流水线理想 $CPI=1$ ，那么：
 - 流水线的实际 $CPI = 1.9$ 。
- ◆ 可采取两种措施来减少分支延迟，**同时采用**
 - 在流水线中尽早判断出分支转移是否成功；
 - 尽早计算出分支目标地址。
- ◆ 下面的讨论中，我们假设：
 - 这两步工作被提前到ID段完成，即分支指令是在ID段的末尾执行完成，所带来的分支延迟为一个时钟周期。

控制冲突-减少分支延迟

◆ 硬件的方法

- 修改数据通路：使得目标地址和分支条件尽早确定
 - 判断是否为0可以在ID段确定，例如：BEQZ, BNEZ
 - 使用另一个加法器计算BTA(分支目标地址)，在ID段生成下一条指令地址，(BTA, PC+4)，选择哪一个取决于ID段确定的CC
 - 必要时使用互锁机制来插入stall

◆ 软件（编译器）的方法

- 预测分支失败
- 预测分支成功
- 延迟分支

◆ 共同点：

- 对分支的处理方法在程序的执行过程中始终是不变的，是静态的。

预测分支失败

◆ 1、预测分支失败

- 允许分支指令后的指令继续在流水线中流动，就好象什么都没发生似的。
- **若确定分支失败**，将分支指令看作是一条普通指令，流水线正常流动。
- **若确定分支成功**，流水线就把在分支指令之后取出的所有指令转化为空操作，并按分支目地重新取指令执行。
- MIPS分支指令平均47%为分支失败

◆ 要保证：

- 分支结果出来之前不会改变处理机的状态，以便一旦猜错时，处理机能够回退到原先的状态。

预测分支失败

◆ 分支失败

分支指令i	IF	ID	EX	MEM	WB				
指令i+1		IF	ID	EX	MEM	WB			
指令i+2			IF	ID	EX	MEM	WB		
指令i+3				IF	ID	EX	MEM	WB	
指令i+4					IF	ID	EX	MEM	WB

◆ 分支成功

分支指令i	IF	ID	EX	MEM	WB				
指令i+1		IF	idle	idle	idle	idle			
分支目标指令			IF	ID	EX	MEM	WB		
分支目标指令+1				IF	ID	EX	MEM	WB	
分支目标指令+2					IF	ID	EX	MEM	WB

控制冲突-减少分支延迟

◆ 2、预测分支成功

- 前题：先知道分支目标地址，后知道分支是否成功。
- 平均53% MIPS分支为分支成功
- **5段流水线中，没有作用**
 - 因为判断分支是否成功与分支目标地址计算在同一个流水段中完成
 - MIPS分支目标地址在ID段才能计算出目标地址
 - MIPS 还是有1个 cycle 的分支延迟

◆ 3、延迟分支

- 从逻辑上“延长”分支指令的执行时间。**把延迟分支看成是由原来的分支指令和若干个延迟槽构成**，不管分支是否成功，都要按顺序执行延迟槽中的指令。
 - 5级流水只需要一个延迟槽就可以确定目标地址和确定条件
 - MIPS 使用这种方式

延迟分支-一个分支延迟槽

◆ 分支失败

分支指令i	IF	ID	EX	MEM	WB				
延迟槽指令i+1		IF	ID	EX	MEM	WB			
指令i+2			IF	ID	EX	MEM	WB		
指令i+3				IF	ID	EX	MEM	WB	
指令i+4					IF	ID	EX	MEM	WB

◆ 分支成功

分支指令i	IF	ID	EX	MEM	WB				
延迟槽指令i+1		IF	ID	EX	MEM	WB			
分支目标指令j			IF	ID	EX	MEM	WB		
分支目标指令j+1				IF	ID	EX	MEM	WB	
分支目标指令j+2					IF	ID	EX	MEM	WB

◆ 分支延迟槽中的指令“掩盖”了流水线原来必须插入的暂停周期

分支延迟指令的调度

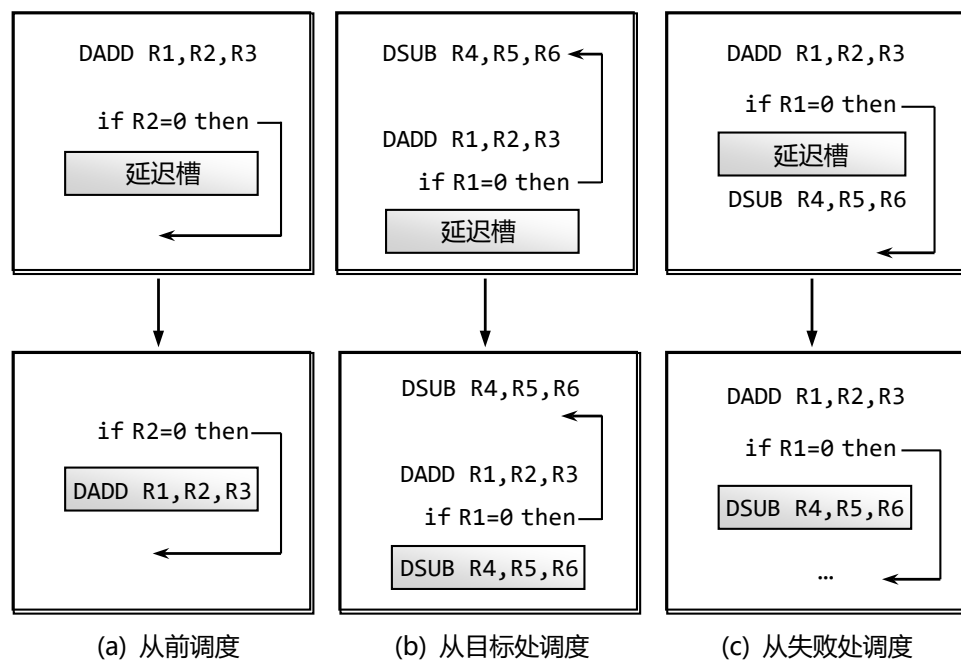
◆ 任务：

- **在延迟槽中放入有用的指令**。由编译器完成。能否带来好处取决于编译器能否把有用的指令调度到延迟槽中。

◆ 三种调度方法：

- 从前调度
 - 分支指令之前的指令，最好
- 从目标处调度
 - 从分支目标地址处取
 - 当分支成功可能性大时，好
- 从失败处调度
 - 当分支失败可能性大时

调度前代码



调度后代码

分支延迟指令的调度

- ◆ 分支延迟受到两个方面的限制：
 - 可以被放入延迟槽中的指令要满足一定的条件；
 - 编译器预测分支转移方向的能力。
- ◆ 进一步改进：**分支取消机制**（取消分支）
 - 当分支的实际执行方向和事先所预测的一样时，执行分支延迟槽中的指令，否则就将分支延迟槽中的指令转化成一个空操作。

“预测成功—取消”分支的执行过程

- ◆ 预测分支成功的情况下，分支取消机制的执行情况
- ◆ 分支成功

分支指令i	IF	ID	EX	MEM	WB				
延迟槽指令i+1		IF	ID	EX	MEM	WB			
分支目标指令j			IF	ID	EX	MEM	WB		
分支目标指令j+1				IF	ID	EX	MEM	WB	
分支目标指令j+2					IF	ID	EX	MEM	WB

- ◆ 分支失败

分支指令i	IF	ID	EX	MEM	WB				
延迟槽指令i+1		IF	idle	idle	idle	idle			
指令i+2			IF	ID	EX	MEM	WB		
指令i+3				IF	ID	EX	MEM	WB	
指令i+4					IF	ID	EX	MEM	WB

评估减少分支策略的效果

- ◆ 加速比 $\text{Pipeline_speedup} = \frac{\text{Pipeline_depth}}{1 + \text{Branch_frequency} \times \text{Branch_penalty}}$
- ◆ Conditional & Unconditional = 14%, 65% change PC

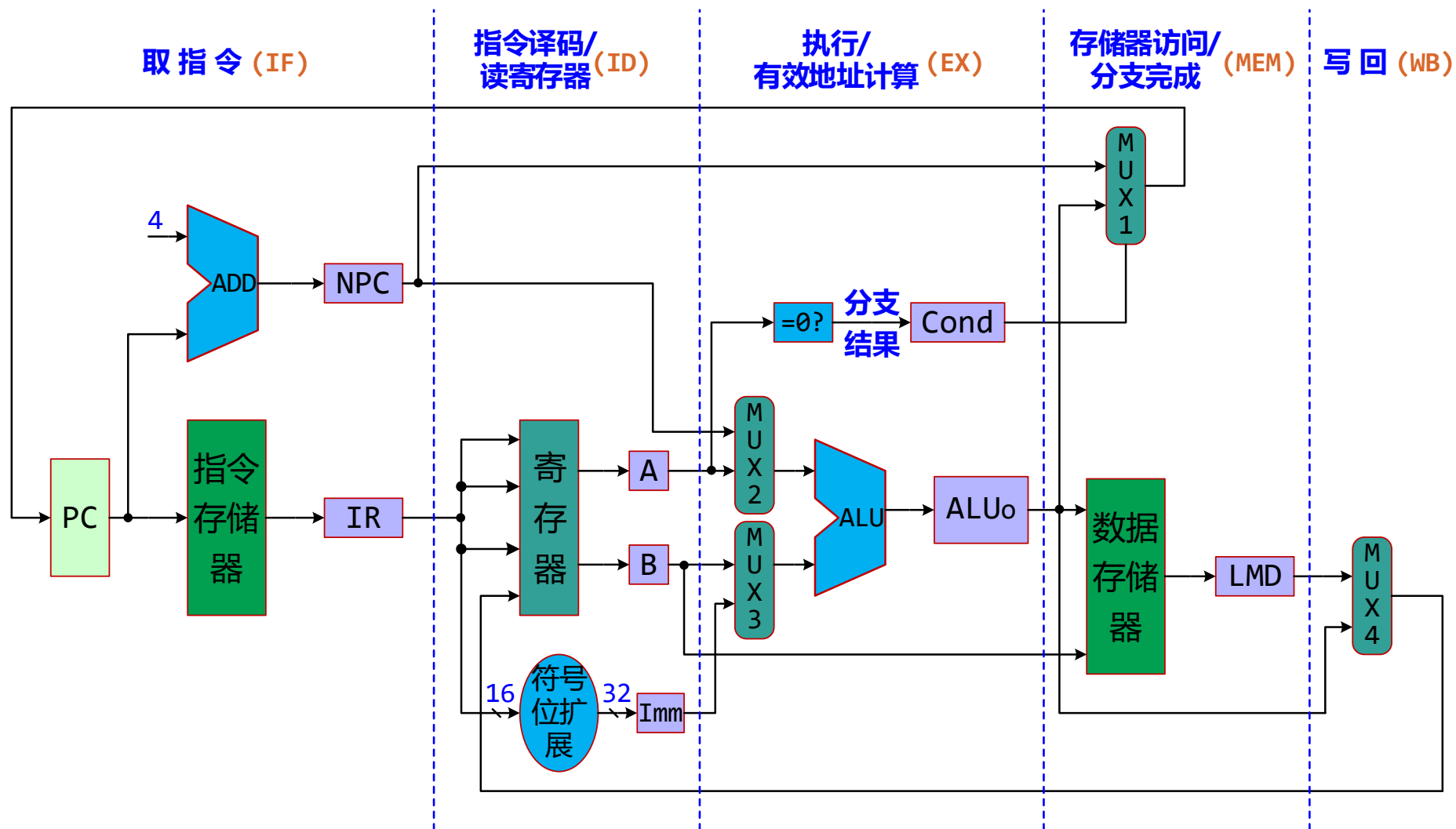
调度策略	分支延迟周期	CPI	加速比 Vs 未使用流水线	加速比 Vs 流水线停顿
流水线停顿	3	1.42	3.5	1.0
分支预测成功	1	1.14	4.4	1.26
分支预测失败	1	1.09	4.5	1.29
延迟分支	0.5	1.07	4.6	1.31

- $1.42 = 1 + 3 * 14\%$
- $1.14 = 1 + 1 * 14\% * 100\%$
- $1.09 = 1 + 1 * 14\% * 65\%$
- $1.07 = 1 + 0.5 * 14\%$

3.5 流水线的实现

- ◆ 3.5.1 MIPS的一种简单实现
- ◆ 实现MIPS指令子集的一种简单数据通路
- ◆ 该数据通路的操作分成5个时钟周期
 - IF: 取指令
 - ID: 指令译码/读寄存器
 - EX: 执行/有效地址计算
 - MEM: 存储器访问/分支完成
 - WB: 写回
- ◆ 只讨论整数指令的实现（包括：load和store，等于0转移，整数ALU指令等。）

实现MIPS指令的一种简单数据通路



3.5.1 MIPS的一种简单实现

- ◆ 一条MIPS指令最多需要以下5个时钟周期：

- ◆ (1) 取指令周期 (IF)

$IR \leftarrow Mem[PC]$

$NPC \leftarrow PC+4$

- ◆ (2) 指令译码/读寄存器周期 (ID)

$A \leftarrow Regs[rs]$

$B \leftarrow Regs[rt]$

$Imm \leftarrow ((IR_{16})^{16} \# \# IR_{16..31})$

- 指令的译码操作和读寄存器操作是并行进行的。
 - 原因：在MIPS指令格式中，操作码字段以及rs、rt字段都是在固定的位置。
- 这种技术称为**固定字段译码技术**

3.5.1 MIPS的一种简单实现

◆ (3) 执行/有效地址计算周期 (EX)

- 不同指令所进行的操作不同:

- 存储器访问指令

$$ALU_0 \leftarrow A + Imm$$

- 寄存器—寄存器ALU指令

$$ALU_0 \leftarrow A \text{ func } B$$

- 寄存器—立即值ALU指令

$$ALU_0 \leftarrow A \text{ op } Imm$$

- 分支指令

$$ALU_0 \leftarrow NPC + (Imm \ll 2) ;$$
$$cond \leftarrow (A == 0)$$

- 将有效地址计算周期和执行周期合并为一个时钟周期，这是因为MIPS指令集采用load / store结构，**没有**任何指令需要同时进行**数据有效地址的计算**、**转移目标地址的计算**和**对数据进行运算**。

3.5.1 MIPS的一种简单实现

◆ (4) 存储器访问/分支完成周期 (MEM)

- 所有指令都要在该周期对PC进行更新。除了分支指令，其他指令都是做 $PC \leftarrow NPC$
- 在该周期内处理的MIPS指令仅仅有load、store和分支三种指令。
- 存储器访问指令 $LMD \leftarrow Mem[ALU_o]$ 或者 $Mem[ALU_o] \leftarrow B$
- 分支指令 $\text{if (cond) } PC \leftarrow ALU_o \text{ else } PC \leftarrow NPC$

◆ (5) 写回周期 (WB)

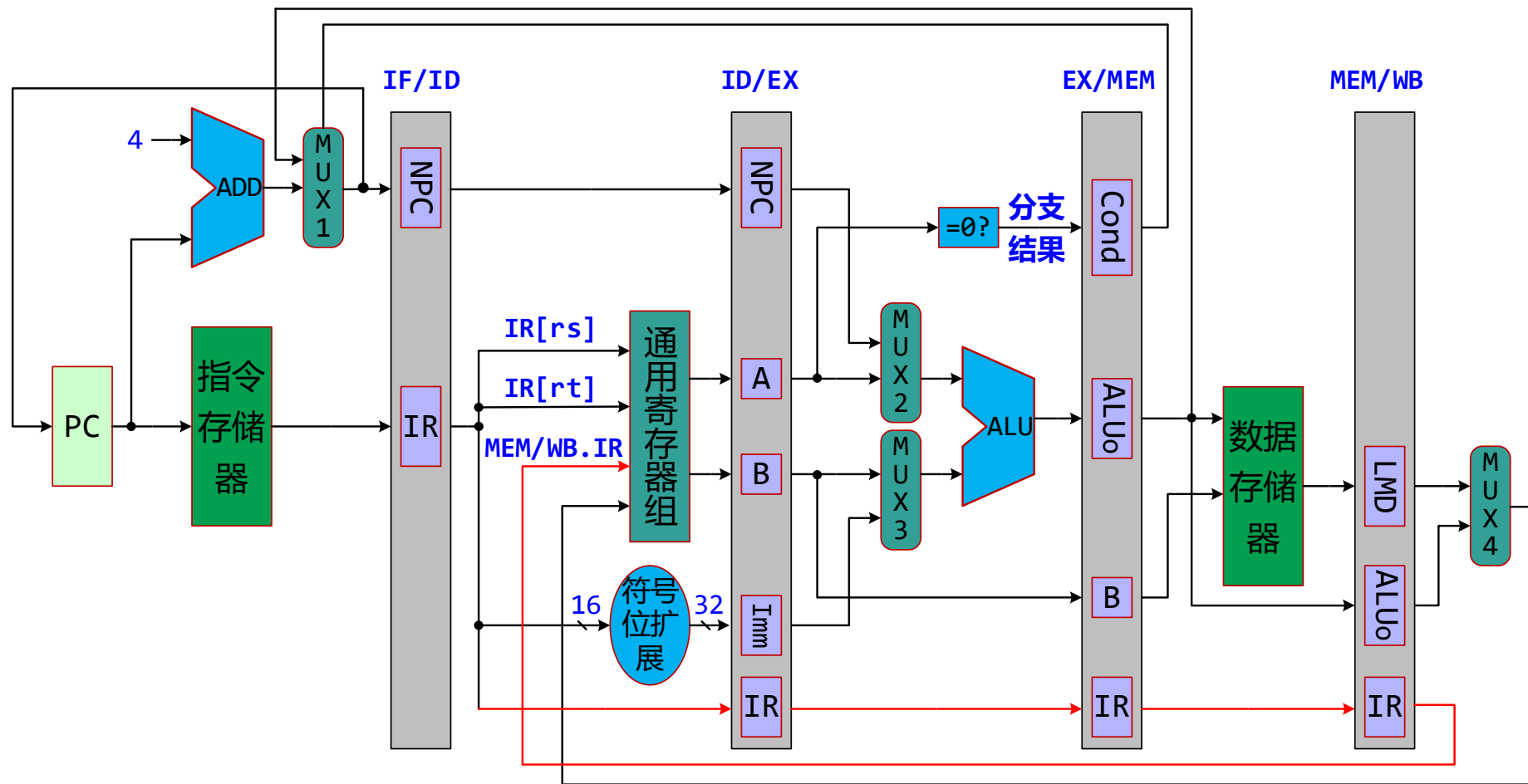
- 不同的指令在写回周期完成的工作也不一样。
- 寄存器—寄存器ALU指令 $Regs[rd] \leftarrow ALU_o$
- 寄存器—立即数ALU指令 $Regs[rt] \leftarrow ALU_o$
- load指令 $Regs[rt] \leftarrow LMD$

3.5.2 基本的MIPS流水线

- ◆ 每一个时钟周期完成的工作看作是流水线的一段，每个时钟周期启动一条新的指令。
- ◆ 1、流水实现的数据通路
 - (1) 设置了流水寄存器
 - 段与段之间设置流水寄存器
 - 流水寄存器的命名：用其相邻的两个段的名称拼合而成
 - 例如：ID段与EX段之间的流水寄存器用ID/EX表示
 - 每个流水寄存器是由若干个寄存器构成的
 - 寄存器的命名形式为：x.y
 - 所包含的字段的命名形式为：x.y[s]
 - ID/EX.IR：流水寄存器ID/EX中的子寄存器IR
 - ID/EX.IR[op]：该寄存器的op字段（即操作码字段）

x：流水寄存器名称
y：具体寄存器名称
s：字段名称

3.5.2 基本的MIPS流水线



流水实现的数据通路

3.5.2 基本的MIPS流水线

◆ 1、流水实现的数据通路

● 流水寄存器的作用

- 将各段的工作隔开，使得它们不会互相干扰；
- 保存相应段的处理结果；
 - ◆ 例如：
 - ◆ EX/MEM.ALU₀：保存EX段ALU的运算结果
 - ◆ MEM/WB.LMD：保存MEM段从数据存储器读出的数据
- 向后传递后面将要用到的数据或者控制信息
 - ◆ 所有有用的数据和控制信息每个时钟周期会随着指令在流水线中的流动往后流动一段。

3.5.2 基本的MIPS流水线

◆ 1、流水实现的数据通路

- (2) 增加了向后传递IR和从MEM/WB.IR回送到通用寄存器组的连接。
- (3) 将对PC的修改移到了IF段，以便PC能及时地加4，为取下一条指令做好准备

◆ 2、每一个流水段进行的操作

$$IR[rs] = IR_{6..10}$$
$$IR[rt] = IR_{11..15}$$
$$IR[rd] = IR_{16..20}$$

流水线的每个流水段的操作

段	所有指令类型		
IF	$\text{IF/ID.IR} \leftarrow \text{Mem}[\text{PC}]$ $\text{IF/ID.NPC, PC} \leftarrow (\text{if}((\text{EX/MEM.IR}[\text{op}] == \text{branch}) \ \& \ \text{EX/MEM.cond}) \{ \text{EX/MEM.ALU}_0 \} \text{ else } \{ \text{PC}+4 \});$		
ID	$\text{ID/EX.A} \leftarrow \text{Regs}[\text{IF/ID.IR}[\text{rs}]]; \text{ID/EX.B} \leftarrow \text{Regs}[\text{IF/ID.IR}[\text{rt}]];$ $\text{ID/EX.NPC} \leftarrow \text{IF/ID.NPC}; \text{ID/EX.IR} \leftarrow \text{IF/ID.IR};$ $\text{ID/EX.Imm} \leftarrow (\text{IF/ID.IR}_{16})^{16} \# \text{IF/ID.IR}_{16..31};$		
	ALU指令	load/store指令	分支指令
EX	$\text{EX/MEM.IR} \leftarrow \text{ID/EX.IR};$ $\text{EX/MEM.ALU}_0 \leftarrow \text{ID/EX.A} \text{ func } \text{ID/EX.B}$ 或 $\text{EX/MEM.ALU}_0 \leftarrow \text{ID/EX.A} \text{ op } \text{ID/EX.Imm};$	$\text{EX/MEM.IR} \leftarrow \text{ID/EX.IR};$ $\text{EX/MEM.ALU}_0 \leftarrow \text{ID/EX.A} + \text{ID/EX.Imm};$ $\text{EX/MEM.B} \leftarrow \text{ID/EX.B};$	$\text{EX/MEM.IR} \leftarrow \text{ID/EX.IR};$ $\text{EX/MEM.ALU}_0 \leftarrow \text{ID/EX.NPC} + \text{ID/EX.Imm} \ll 2;$ $\text{EX/MEM.cond} \leftarrow (\text{ID/EX.A} == 0);$

流水线的每个流水段的操作

段	所有指令类型		
	ALU指令	load/store指令	分支指令
MEM	$\text{MEM/WB.IR} \leftarrow \text{EX/MEM.IR};$ $\text{MEM/WB.ALU}_0 \leftarrow \text{EX/MEM.ALU}_0;$	$\text{MEM/WB.IR} \leftarrow \text{EX/MEM.IR};$ $\text{MEM/WB.LMD} \leftarrow \text{Mem}[\text{EX/MEM.ALU}_0];$ 或 $\text{Mem}[\text{EX/MEM.ALU}_0] \leftarrow \text{EX/MEM.B};$	
WB	$\text{Regs}[\text{MEM/WB.IR}[\text{rd}]] \leftarrow \text{MEM/WB.ALU}_0;$ 或 $\text{Regs}[\text{MEM/WB.IR}[\text{rt}]] \leftarrow \text{MEM/WB.ALU}_0;$	$\text{Regs}[\text{MEM/WB.IR}[\text{rt}]] \leftarrow \text{MEM/WB.LMD};$	

3.5.2 基本的MIPS流水线

◆ 3、流水线的控制

- 主要是如何控制4个多路选择器。
- **MUX2**: 若ID/EX.IR中的指令是分支指令, 则选择ID/EX.NPC, 否则选ID/EX.A。
- **MUX3**: 若ID/EX.IR中的指令是寄存器—寄存器型ALU指令, 则选ID/EX.B, 否则选ID/EX.Imm。
- **MUX1**: 若EX/MEM.IR中的指令是分支指令, 而且EX/MEM.cond为真, 则选EX/MEM.ALU₀, 即分支目标地址, 否则选PC+4。
- **MUX4**: 若MEM/WB.IR中的指令是load指令, 则选MEM/WB.LMD, 否则选MEM/WB.ALU₀。

3.5.2 基本的MIPS流水线

- **第5个多路器**：从MEM/WB回传至通用寄存器组的写入
 - 地址应该是从MEM/WB.IR[rd] 和MEM/WB.IR[rt]中选一个
 - 寄存器—寄存器型ALU指令：选择MEM/WB.IR[rd] ；
 - 寄存器—立即数型ALU指令和load指令：选择MEM/WB.IR[rt] 。

◆ 解决数据冲突的问题

- 1) 所有的数据冲突均可以在ID段检测到
 - 如果存在数据冲突，就在相应的指令流出ID段之前将之暂停。
 - 完成该工作的硬件称为流水线的互锁机制。
- 2) 在ID段确定需要什么样的定向，并设置相应的控制
 - 降低流水线的硬件复杂度。不必挂起已经改变了机器状态的指令
- 3) 也可以在使用操作数的那个时钟周期的开始检测冲突和确定必需的定向。
- **检测冲突是通过比较寄存器地址是否相等来实现的**

3.5.2 基本的MIPS流水线

- ◆ 举例： load互锁
 - 由于使用load的结果而引起的流水线互锁称为load互锁
- ◆ 在ID段检测是否存在RAW冲突
 - 这时load指令在EX段

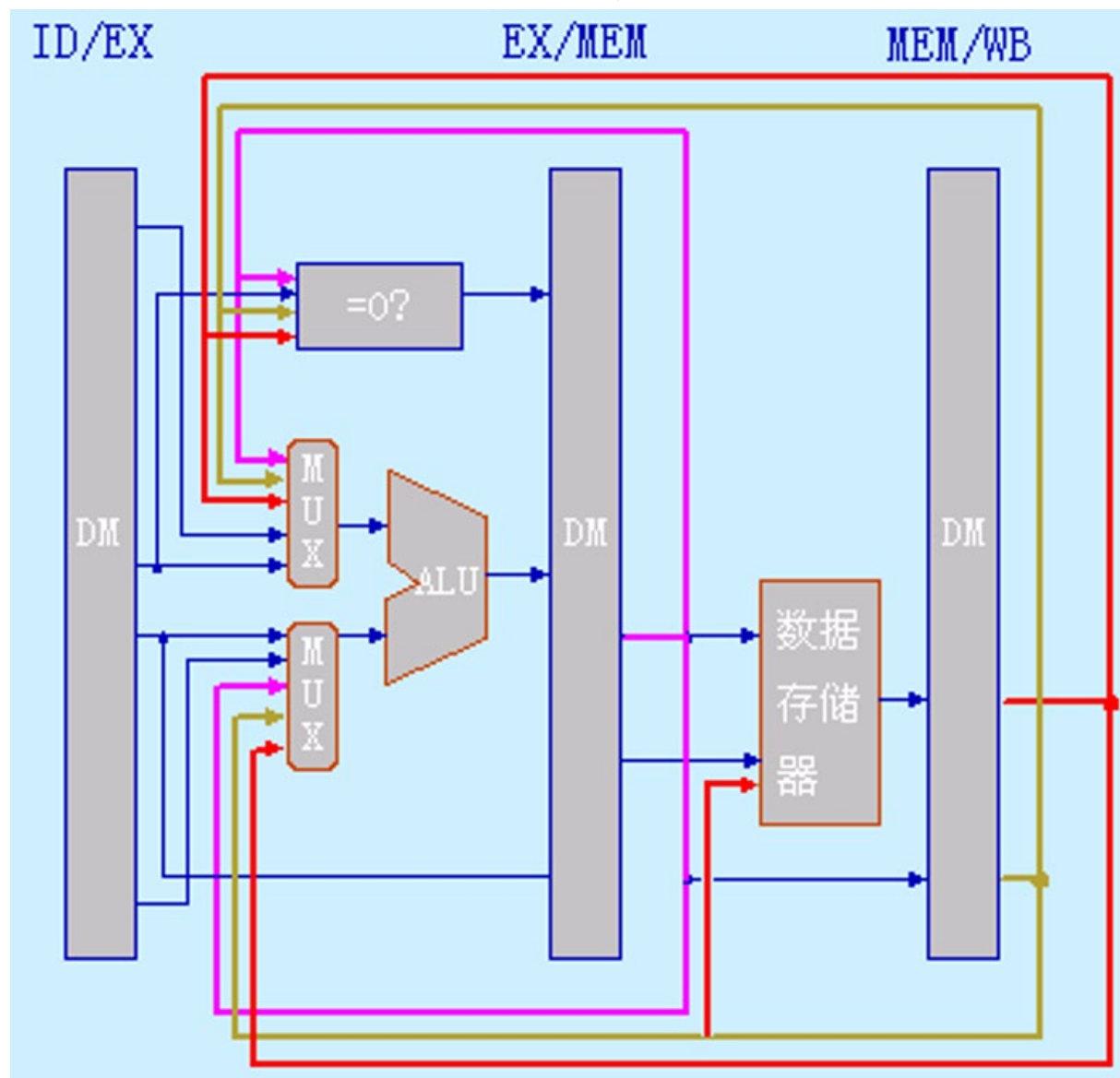
ID/EX中的操作码 (ID/EX.IR[op])	IF/ID中的操作码 (IF/ID.IR[op])	比较的操作数字段
load	RR ALU	ID/EX.IR[rt]=IF/ID.IR[rs]
load	RR ALU	ID/EX.IR[rt]=IF/ID.IR[rt]
load	load、store ALU立即数或分支	ID/EX.IR[rt]=IF/ID.IR[rs]

3.5.2 基本的MIPS流水线

- ◆ **若检测到RAW冲突**，流水线互锁机制必须在流水线中插入停顿，并使当前正处于IF段和ID段的指令不再前进。
 - 将ID/EX.IR中的操作码改为全0，（全0表示空操作）
 - IF/ID寄存器的内容回送到自己的入口
- ◆ **定向逻辑**
 - 通过比较流水寄存器中的寄存器地址来确定
 - 若 $(ID/EX.IR.op == RR_ALU) \& (EX/MEM.IR.op == RR_ALU) \& (ID/EX.IR[rt] == EX/MEM.IR[rd])$ ，
则 EX/MEM.ALU₀定向到ALU的下面一个输入。
 - 若 $(ID/EX.IR[op] == RR_ALU) \& (MEM/WB.IR[op] == load) \& (ID/EX.IR[rt] == MEM/WB.IR[rt])$ ，
则 把MEM/WB.LMD定向到ALU的下面一个输入。

3.5.2 基本的MIPS流水线

- ◆ 流水线增设的定向路径

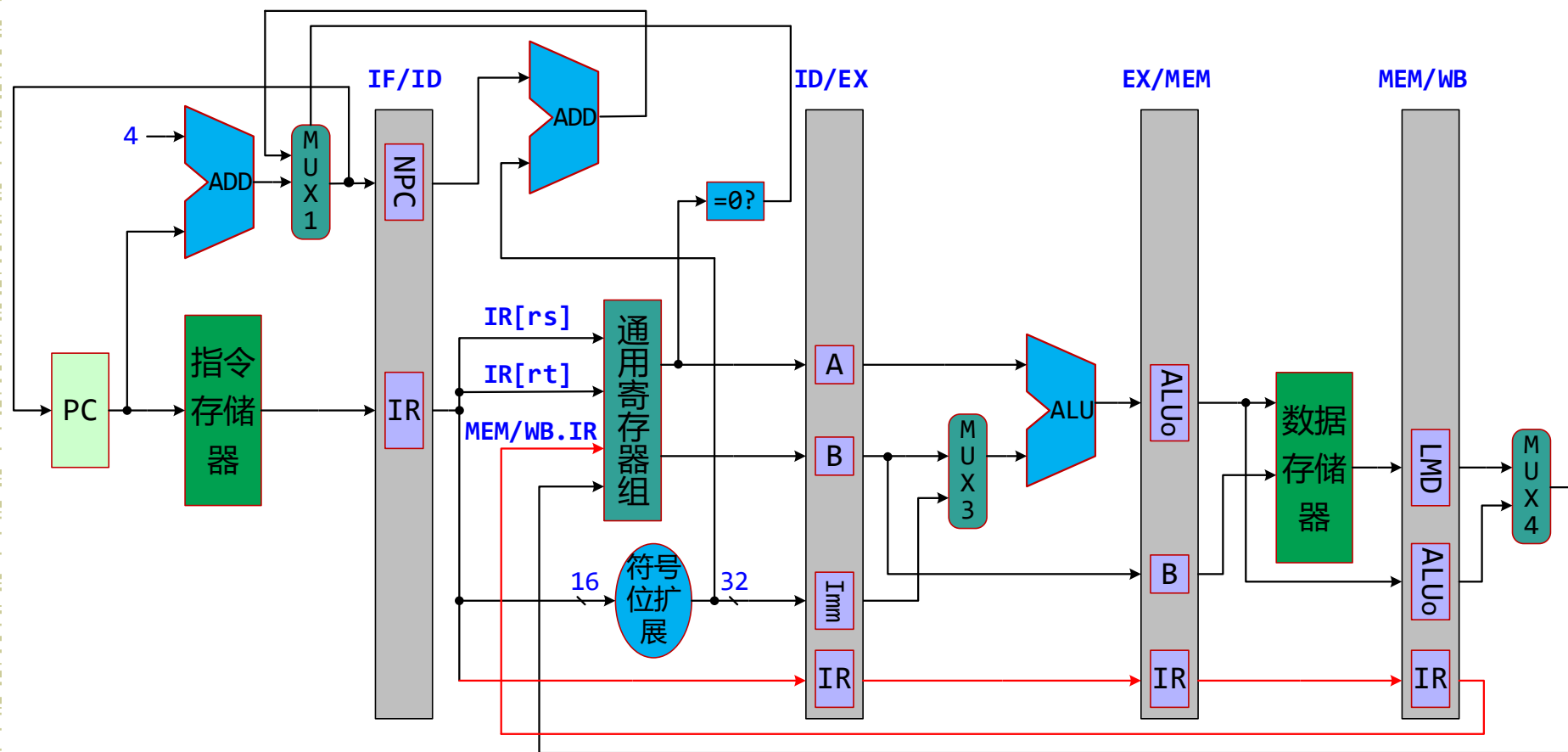


3.5.2 基本的MIPS流水线

◆ 4、控制冲突

- 分支指令的条件测试和分支目标地址计算在EX段完成，对PC的修改在MEM段完成。
- 它所带来的分支延迟是3个时钟周期。
- 减少分支延迟：（把上述工作提前到ID段进行）
 - 在ID段增设一个加法器，用于计算分支目标地址。
 - 把条件测试“=0?”的逻辑电路移到ID段。
 - 这些结果直接回送到IF段的MUX1。
 - 改进后的流水线对分支指令的处理。

3.5.2 基本的MIPS流水线



为减少分支延迟，改进后的流水线数据通路

3.5.2 基本的MIPS流水线

◆ 改进后流水线的分支操作

流水段	分支指令操作
IF	$\begin{aligned} & \text{IF/ID.IR} \leftarrow \text{Mem}[\text{PC}]; \\ & \text{IF/ID.NPC, PC} \leftarrow \\ & \quad (\text{if}((\text{IF/ID.IR}[\text{op}] == \text{branch}) \ \& \ ((\text{Regs}[\text{IF/ID.IR}[\text{rs}]] == 0))) \\ & \quad \quad \{ \text{IF/ID.NPC} + (\text{IF/ID.IR}_{16})^{16} \ \#\# \ (\text{IF/ID.IR}_{16..31} \ll 2) \} \\ & \quad \text{else} \ \{ \text{PC} + 4 \}); \end{aligned}$
ID	$\begin{aligned} & \text{ID/EX.A} \leftarrow \text{Regs}[\text{IF/ID.IR}[\text{rs}]]; \quad \text{ID/EX.B} \leftarrow \text{Regs}[\text{IF/ID.IR}[\text{rt}]]; \\ & \text{ID/EX.IR} \leftarrow \text{IF/ID.IR}; \\ & \text{ID/EX.Imm} \leftarrow (\text{IF/ID.IR}_{16})^{16} \ \#\# \ \text{IF/ID.IR}_{16..31}; \end{aligned}$
EX	
MEM	
WB	